Hierarchical k-GNN Explanations via Generative Interpretation Networks

Comparing 1-GNN and 1-2-GNN for Model-Level Graph Classification Interpretability

[Author Name]

March 2026

Abstract

Higher-order k-GNNs provably increase expressive power beyond the Weisfeiler–Leman hierarchy, but it is unclear whether this additional expressiveness leads the model to learn qualitatively different internal representations of graph classes, or merely the same representations computed more accurately. We investigate this question by comparing a standard 1-GNN with a hierarchical 1-2-GNN using the GIN-Graph framework for model-level explanation generation on the MUTAG and PROTEINS benchmarks. We contribute two technical pieces needed to make the comparison possible: a fully differentiable dense wrapper that preserves gradient flow through pair-level aggregation in the 1-2-GNN, and a correction to the validation metric that measures actual cosine similarity between generated and real graph embeddings rather than reusing prediction probability as a proxy. Across 500-graph population comparisons, the two architectures produce different class prototype families even when their test accuracies match. On MUTAG (89.5% for both), the 1-GNN favours N/O-rich mutagens while the 1-2-GNN favours halogen-heavy ones. On PROTEINS, the 1-GNN gives the strongest Non-Enzyme scores while the 1-2-GNN gives the closest population-level node-type match and the highest Enzyme validity. Cross-model classification reveals sharp asymmetries: generated graphs that one model labels correctly are often rejected by the other (12.4% transfer on MUTAG 1-2-GNN Non-Mutagen; 9.4% on PROTEINS 1-GNN Non-Enzyme). After adapting GIN-Graph to the hierarchical 1-2-GNN and correcting the validation metric, we find that 1-GNN and 1-2-GNN learn different class prototype families. Higher-order message passing changes *what* is represented, not uniformly *how well*.

Contents

1 Introduction

Graph Neural Networks (GNNs) have become the standard approach for learning on graph-structured data, achieving strong results on tasks ranging from molecular property prediction to social network analysis. Despite their effectiveness, GNNs suffer from the same interpretability challenges as other deep learning models: their predictions are difficult to explain in human-understandable terms.

Standard message-passing GNNs are bounded by the 1-WL graph isomorphism test [Morris et al.(2019)]. Morris et al. [Morris et al.(2019)] proposed k-GNNs over k-element node subsets. These models match the expressiveness of the k-WL test and can therefore distinguish graph structures that standard GNNs cannot.

A natural question arises: *does this additional structural expressiveness lead the model to learn different internal representations of graph classes, and can we observe these differences through generated explanations?* If a model captures richer structural patterns, the explanations it produces (representative graphs that characterise each class) may reveal what structural features each architecture has learned to associate with each class.

We investigate this question using the GIN-Graph framework [Sun et al.(2025)], which generates model-level explanation graphs through a GAN-based approach guided by a pretrained classifier. Model-level explanations aim to answer the question "what does a typical graph of class c look like according to the model?" by generating new graphs that maximise the model's confidence for that class while remaining structurally realistic. This contrasts with instance-level methods that explain individual predictions by identifying important subgraphs.

Specifically, we compare:

- A **1-GNN**: standard node-level message passing, equivalent to 1-WL;
- A **1-2-GNN**: hierarchical architecture combining node-level (1-GNN) and pairwise (2-GNN) message passing, achieving expressiveness beyond 1-WL.

This thesis makes three contributions:

1. Reuses the dense 1-GNN wrapper from GIN-Graph [Sun et al.(2025)] and applies the same logic to the 2-GNN branch of a 1-2-GNN classifier, so that GIN-Graph training can be run against a hierarchical k-GNN.
2. Computes the embedding similarity component s of the validation score via cosine similarity between generated and real graph embeddings. The GIN-Graph reference implementation uses the prediction probability p as a proxy for s; we use the actual embedding.
3. Compares the explanation prototypes produced by the two architectures on MUTAG and PROTEINS. At matched classifier accuracy the models generate qualitatively different class prototype families (e.g. N/O-rich versus halogen-heavy mutagens on MUTAG), and cross-model classification of 500 generated graphs per class reveals sharp asymmetries (12.4% transfer on MUTAG 1-2-GNN Non-Mutagen; 9.4% on PROTEINS 1-GNN Non-Enzyme), supporting a case- and dataset-dependent reading of higher-order message passing rather than blanket superiority.

The main empirical claim is simple: if two architectures reach similar classification accuracy but generate different class-conditional prototype datasets, then higher-order message passing is changing the model's internal class representation rather than just im-

proving the same one.

2 Background

2.1 Graph Neural Networks

A graph $G = (V, E)$ consists of a node set V with $|V| = n$ and an edge set $E \subseteq V \times V$. Each node $v \in V$ carries a feature vector $\mathbf{x}_v \in \mathbb{R}^d$. We denote the adjacency matrix as $\mathbf{A} \in \{0,1\}^{n \times n}$ where $A_{uv} = 1$ if $(u, v) \in E$, and the feature matrix as $\mathbf{X} \in \mathbb{R}^{n \times d}$ where row v is $\mathbf{x}_v$.

GNNs learn node representations through iterative *message passing*. At each layer t, a node updates its representation by aggregating information from its neighbours:

$$\mathbf{h}_v^{(t)} = \mathrm{UPDATE}\big(\mathbf{h}_v^{(t-1)},\ \mathrm{AGGREGATE}\big(\{\!\{\mathbf{h}_u^{(t-1)} : u \in \mathcal{N}(v)\}\!\}\big)\big)\,, \tag{1}$$

where $\mathcal{N}(v) = \{u \in V : (u, v) \in E\}$ denotes the neighbourhood of v, $\mathbf{h}_v^{(0)} = \mathbf{x}_v$ is the initial node feature, and $\{\!\{\cdot\}\!\}$ denotes a multiset.

The choice of UPDATE and AGGREGATE functions determines the specific GNN variant. In our implementation, these are parameterised by separate linear transformations for the self-feature and the aggregated neighbour features. After T layers of message passing, a *readout* function pools all node embeddings into a single graph-level representation:

$$\mathbf{h}_G = \mathrm{READOUT}\big(\{\mathbf{h}_v^{(T)} : v \in V\}\big)\,. \tag{2}$$

Common choices for READOUT include sum, mean, and max pooling over the node set. We use sum pooling for the classifier readouts, as it preserves information about both the features and the number of nodes.

2.2 The k-WL Hierarchy and k-GNNs

The *Weisfeiler–Leman* (WL) graph isomorphism test is a classical algorithm that iteratively refines node colour labels based on the multiset of neighbour colours. Two graphs are distinguished if, after some number of iterations, they produce different multisets of colours. The 1-WL test operates on individual nodes.

A fundamental result in GNN theory is that standard message-passing GNNs are *at most* as powerful as the 1-WL test [Morris et al.(2019)]: if 1-WL cannot distinguish two graphs, no message-passing GNN can either. This means there exist structurally different graphs (e.g., certain regular graphs) that no standard GNN can tell apart.

The k-WL test generalises this by operating on k-tuples of nodes, achieving strictly increasing discriminative power: for all $k \geq 2$, $(k+1)$-WL is strictly more powerful than k-WL [Cai et al.(1992)]. Morris et al. [Morris et al.(2019)] proposed k-GNNs that achieve this higher expressiveness by performing message passing on k-element subsets of nodes ("k-sets") rather than individual nodes.

For a k-set $s = \{v_1, \ldots, v_k\} \subseteq V$, the *local neighbourhood* is:

$$\mathcal{N}_L(s) = \{s' \subseteq V : |s'| = k,\ |s \triangle s'| = 2,\ \text{the differing elements are adjacent in } G\}\,, \tag{3}$$

where $s \triangle s'$ denotes the symmetric difference. In words: a neighbour of s is obtained by replacing exactly one element of s with a new node that is adjacent (in the original graph G) to the removed node.

Example. Consider a 2-set $s = \{u, v\}$. Its neighbours are all pairs $\{v, w\}$ where $(u, w) \in E$ (replacing u with adjacent w) and all pairs $\{u, w\}$ where $(v, w) \in E$ (replacing v with adjacent w). This means the 2-GNN can capture pairwise relationships and bond-level patterns that are invisible to a 1-GNN operating on individual nodes.

2.3 GIN-Graph

GIN-Graph [Sun et al.(2025)] is a model-level explanation method that generates class-representative graphs using a generative adversarial approach. The key idea is to train a generator $\mathcal{G}$ to produce graphs that satisfy two objectives simultaneously:

1. **Realism**: generated graphs should be structurally similar to real graphs from the dataset, enforced by a WGAN-GP discriminator.
2. **Class specificity**: generated graphs should be confidently classified as the target class by the pretrained GNN, enforced by a classification guidance loss.

The generator maps noise $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ to a graph $(\tilde{A}, \tilde{X})$ using Gumbel-Softmax for differentiable discrete sampling. The training objective balances the two losses through a dynamic weighting schedule that shifts focus from realism (early training) to class specificity (late training).

3 Method

This section details the mathematical formulations of our two classifier architectures (1-GNN and 1-2-GNN), the GIN-Graph generator, the training procedure, and the differentiable dense wrapper that bridges them.

3.1 1-GNN Architecture

Our 1-GNN implements message passing with separate transformations for self-features and neighbour-aggregated features. At layer t, the update rule for node v is:

$$\mathbf{h}_v^{(t)} = \sigma\left(\mathbf{h}_v^{(t-1)}\mathbf{W}_1^{(t)} + \sum_{u \in \mathcal{N}(v)} \mathbf{h}_u^{(t-1)}\mathbf{W}_2^{(t)}\right), \tag{4}$$

where $\mathbf{W}_1^{(t)}, \mathbf{W}_2^{(t)} \in \mathbb{R}^{d_{t-1} \times d_t}$ are learnable weight matrices, σ is the ReLU activation function $\sigma(x) = \max(0, x)$, and $d_0 = d$ (the input feature dimension). The self-transformation $\mathbf{h}_v^{(t-1)}\mathbf{W}_1^{(t)}$ allows the model to retain and transform the node's own representation, while the neighbour term $\sum_{u \in \mathcal{N}(v)} \mathbf{h}_u^{(t-1)}\mathbf{W}_2^{(t)}$ aggregates structural context.

This can equivalently be written in matrix form for the entire graph:

$$\mathbf{H}^{(t)} = \sigma\Big(\mathbf{H}^{(t-1)}\mathbf{W}_1^{(t)} + \mathbf{A}\,\mathbf{H}^{(t-1)}\mathbf{W}_2^{(t)}\Big), \tag{5}$$

where $\mathbf{H}^{(t)} \in \mathbb{R}^{n \times d_t}$ is the matrix of all node representations at layer t, and $\mathbf{A} \in \{0,1\}^{n \times n}$ is the adjacency matrix. The matrix product $\mathbf{A}\,\mathbf{H}^{(t-1)}$ computes the sum of neighbour features for every node simultaneously.

After $T = 3$ layers with hidden dimension $d_1 = d_2 = d_3 = 64$, the graph embedding is obtained via sum pooling:

$$\mathbf{h}_G^{(1)} = \sum_{v \in V} \mathbf{h}_v^{(T)} \in \mathbb{R}^{d_T}. \tag{6}$$

This embedding is fed through a two-layer classifier MLP with ReLU activation and dropout:

$$\hat{y} = \mathrm{softmax}\Big(\mathbf{W}_o\, \sigma(\mathbf{W}_c\, \mathbf{h}_G^{(1)})\Big), \tag{7}$$

where $\mathbf{W}_c \in \mathbb{R}^{d_T \times d_T}$ and $\mathbf{W}_o \in \mathbb{R}^{d_T \times C}$ with C being the number of classes.

3.2 2-GNN and the Hierarchical 1-2-GNN

The 2-GNN operates on 2-sets (unordered node pairs) rather than individual nodes. For each pair $s = \{u, v\}$ with $u < v$ (canonical ordering), the initial feature vector incorporates the 1-GNN embeddings of both nodes and an *isomorphism type* indicator:

$$\mathbf{f}_s^{(0)} = \big[\mathbf{h}_u^{(T)} \parallel \mathbf{h}_v^{(T)} \parallel \mathbb{1}[(u,v) \in E]\big] \in \mathbb{R}^{2d_T+1}, \tag{8}$$

where $\parallel$ denotes vector concatenation. The isomorphism type $\mathbb{1}[(u,v) \in E] \in \{0,1\}$ indicates whether u and v are connected by an edge. This allows the 2-GNN to distinguish between pairs of connected nodes and pairs of unconnected nodes, a distinction that carries structural meaning (e.g., in a molecule, bonded vs. non-bonded atom pairs).

Message passing on 2-sets follows the neighbourhood definition from Equation (3). For a 2-set $s = \{u, v\}$, the local neighbourhood decomposes into two cases:

$$\mathcal{N}_L(\{u,v\}) = \underbrace{\{\{v,w\} : w \neq u,\ (u,w) \in E\}}_{\text{Case 1: replace } u \text{ with } w} \cup \underbrace{\{\{u,w\} : w \neq v,\ (v,w) \in E\}}_{\text{Case 2: replace } v \text{ with } w}. \tag{9}$$

In Case 1, we remove u from the pair and add a new node w that must be adjacent to u in the original graph. In Case 2, we remove v and add w adjacent to v. This ensures that neighbourhood relationships in the 2-GNN reflect the edge structure of the underlying graph.

The 2-GNN layer update rule mirrors the 1-GNN structure:

$$\mathbf{f}_s^{(\ell+1)} = \sigma\left(\mathbf{f}_s^{(\ell)}\mathbf{W}_3^{(\ell)} + \sum_{s' \in \mathcal{N}_L(s)} \mathbf{f}_{s'}^{(\ell)}\mathbf{W}_4^{(\ell)}\right), \tag{10}$$

where $\mathbf{W}_3^{(\ell)}, \mathbf{W}_4^{(\ell)}$ are the 2-GNN weight matrices at layer ℓ. We apply $T_2 = 2$ layers.

The **hierarchical 1-2-GNN** combines both levels. It first runs the 1-GNN (Equation 4) for $T_1 = 3$ layers to obtain node embeddings, then uses these embeddings to initialise 2-GNN features (Equation 8), and runs $T_2 = 2$ layers of 2-GNN message passing. The final graph representation concatenates both readouts:

$$\mathbf{h}_G = \Big[\mathbf{h}_G^{(1)} \parallel \mathbf{h}_G^{(2)}\Big] \in \mathbb{R}^{2d_T}, \tag{11}$$

where the 1-GNN readout is $\mathbf{h}_G^{(1)} = \sum_{v \in V} \mathbf{h}_v^{(T_1)}$ and the 2-GNN readout pools over all pairs:

$$\mathbf{h}_G^{(2)} = \sum_{\{u,v\} \in \binom{V}{2}} \mathbf{f}_{\{u,v\}}^{(T_2)} \in \mathbb{R}^{d_T}. \tag{12}$$

The concatenated embedding $\mathbf{h}_G$ is then fed through the same classifier structure as Equation (7), but with input dimension $2d_T$ instead of d_T.

Scalability. The number of 2-sets grows as $\binom{n}{2} = \frac{n(n-1)}{2}$, which becomes prohibitive for large graphs. For PROTEINS (up to 620 nodes, yielding up to 191,890 pairs), we process all pairs using a numba-accelerated sparse kernel with CSR adjacency representation, achieving efficient $O(\text{degree})$ neighbour lookup instead of $O(N)$ dense masks. This enables exact computation over the full pairwise structure without sampling.

3.3 GIN-Graph Generator

The generator $\mathcal{G}$ maps a latent noise vector $\mathbf{z} \in \mathbb{R}^{d_z}$ to a graph $(\tilde{A}, \tilde{X})$ through a backbone MLP followed by separate output heads:

$$\mathbf{h} = \text{LeakyReLU}(\mathbf{W}_2 \, \text{LeakyReLU}(\mathbf{W}_1 \mathbf{z})) \in \mathbb{R}^{2d_h}, \tag{13}$$

$$\tilde{A}_{\text{raw}} = \text{sym}(\text{reshape}(\mathbf{h}\,\mathbf{W}_A,\ N \times N)), \tag{14}$$

$$\tilde{X}_{\text{raw}} = \text{reshape}(\mathbf{h}\,\mathbf{W}_X,\ N \times D), \tag{15}$$

where $d_z = 32$ is the latent dimension, $d_h = 128$ is the hidden dimension, N is the maximum number of nodes, D is the number of node feature types, and $\text{sym}(\mathbf{M}) = \frac{1}{2}(\mathbf{M} + \mathbf{M}^\top)$ ensures the adjacency logits are symmetric (undirected graphs).

Gumbel-Softmax. Graphs are inherently discrete objects: edges are either present or absent, and node types are categorical. Standard gradient-based optimisation cannot backpropagate through discrete sampling. The Gumbel-Softmax trick [Jang et al.(2017)] provides a differentiable relaxation.

For a categorical distribution with unnormalised log-probabilities (logits) $\pi_1, \ldots, \pi_K$, the Gumbel-Softmax sample is:

$$y_i = \frac{\exp((\pi_i + g_i)/\tau)}{\sum_{j=1}^{K} \exp((\pi_j + g_j)/\tau)}, \quad g_i = -\log(-\log(u_i)), \quad u_i \sim \text{Uniform}(0,1), \tag{16}$$

where $\tau > 0$ is a temperature parameter. As $\tau \to 0$, the output approaches a one-hot vector (hard categorical sample); as $\tau \to \infty$, it approaches a uniform distribution. During training, we use the *straight-through estimator*: the forward pass produces hard (discrete) samples via $\arg\max$, but the backward pass uses the continuous Gumbel-Softmax gradients. This gives the generator discrete outputs while maintaining gradient flow.

For the **adjacency matrix**, each potential edge (i, j) is a binary choice (edge/no-edge). We construct 2-class logits $[\tilde{A}_{\text{raw}}[i,j],\ -\tilde{A}_{\text{raw}}[i,j]]$ and apply Gumbel-Softmax with $K = 2$. To guarantee symmetry ($\tilde{A}_{ij} = \tilde{A}_{ji}$), we sample only the upper triangle and mirror it:

$$\tilde{A}_{\text{upper}} = \text{triu}(\text{GumbelSoftmax}(\tilde{A}_{\text{raw}}, \tau)), \quad \tilde{A} = \tilde{A}_{\text{upper}} + \tilde{A}_{\text{upper}}^\top. \tag{17}$$

For **node features**, each node has D possible types (e.g., 7 atom types for MUTAG). We apply Gumbel-Softmax with $K = D$ to produce one-hot feature vectors:

$$\tilde{X}[v,:] = \text{GumbelSoftmax}(\tilde{X}_{\text{raw}}[v,:],\ \tau) \in \{0,1\}^D. \tag{18}$$

The temperature $\tau = 1.0$ during training (allowing exploration) and $\tau = 0.1$ during evaluation (producing sharper, more discrete outputs).

3.4 Training Objective

The total generator loss combines adversarial, GNN-guidance, and structural regularisation terms:

$$\mathcal{L}_G = (1 - \lambda(t))\,\mathcal{L}_{\text{GAN}} + \lambda(t)\,\mathcal{L}_{\text{GNN}} + \mathcal{L}_{\text{reg}}, \tag{19}$$

where $\lambda(t) \in [0, 1]$ is a dynamic weight that increases over training, and $\mathcal{L}_{\text{reg}}$ comprises structural regularisation losses described below. The three terms correspond to three goals: making the generated graphs look realistic ($\mathcal{L}_{\text{GAN}}$), making them belong to the target class ($\mathcal{L}_{\text{GNN}}$), and making them the right size and composition ($\mathcal{L}_{\text{reg}}$). We now explain each in turn, beginning with the adversarial component and the discriminator that drives it.

3.4.1 The Discriminator

The discriminator $\mathcal{D}$ is a graph neural network that maps a graph to a single scalar. It operates on dense batched inputs (node features $\mathbf{X} \in \mathbb{R}^{B\times N\times D}$ and adjacency $\mathbf{A} \in [0, 1]^{B\times N\times N}$) so that the same forward pass handles both real and generated graphs. The architecture has three stages:

1. **Two dense GCN layers** with hidden dimension 128 and ReLU activations. Each layer computes $\mathbf{H}^{(t)} = \sigma(\hat{\mathbf{A}}\,\mathbf{H}^{(t-1)}\,\mathbf{W}^{(t)})$, where $\hat{\mathbf{A}}$ is the adjacency with self-loops added and symmetric degree normalisation applied.
2. **Mean pooling** across the node dimension: $\mathbf{h}_G = \frac{1}{N}\sum_{v=1}^{N} \mathbf{h}_v^{(2)} \in \mathbb{R}^{128}$. This produces a single graph-level vector per graph.
3. **Two-layer MLP** with LeakyReLU activation: $\mathcal{D}(G) = \mathbf{w}^\top \text{LeakyReLU}_{0.2}(\mathbf{W}\,\mathbf{h}_G) \in \mathbb{R}$.

Higher values of $\mathcal{D}(G)$ indicate that the graph appears real; lower values indicate that it appears generated. There is no sigmoid or softmax at the output, so $\mathcal{D}(G)$ is not a probability. In a standard binary classifier, the scalar would pass through a sigmoid $\sigma(x) = 1/(1 + e^{-x})$ to produce a value in $[0, 1]$. Here the output is left unbounded. The reason is explained below in the comparison with vanilla GANs.

The discriminator is distinct from the pretrained k-GNN classifier Φ used in $\mathcal{L}_{\text{GNN}}$: $\mathcal{D}$ is trained from scratch during GIN-Graph training and scores graphs on realism, while Φ is frozen and scores graphs on class membership.

3.4.2 The Two-Player Setup

The generator $\mathcal{G}$ and the discriminator $\mathcal{D}$ are trained in alternating steps:

- The **discriminator** receives a batch of real graphs from the training set and a batch of generated graphs from the current generator, and updates its weights to increase scores on real graphs and decrease scores on generated graphs.
- The **generator** produces a batch of graphs, passes them through the discriminator, and updates its weights to increase the scores the discriminator assigns.

Neither network is trained to convergence; they co-evolve. In each training step, the discriminator updates once, then the generator updates once ($n_{\text{critic}} = 1$). This alternation is necessary. If $\mathcal{D}$ were trained to convergence against a fixed generator, its output would be correct (low on all generated graphs, high on all real graphs) but uninformative as a gradient signal: the generated region of input space would become flat, so small changes to a generated graph would not change $\mathcal{D}$'s score, and the generator would receive no direction for improvement. Keeping $\mathcal{D}$ partially trained ensures that its scores still vary across the region where the generator's current outputs lie.

3.4.3 Discriminator Loss

With the setup in place, we can state the discriminator loss. In each training step, we draw a batch of $B = 64$ real graphs G from the training set and a batch of B fake graphs $\tilde{G}$ from the generator. The discriminator minimises:

$$\mathcal{L}_D = \underbrace{\mathbb{E}_{\tilde{G}\sim\mathcal{G}}[\mathcal{D}(\tilde{G})]}_{\text{score on fake}} - \underbrace{\mathbb{E}_{G\sim p_{\text{data}}}[\mathcal{D}(G)]}_{\text{score on real}} + \underbrace{\lambda_{\text{GP}}\,\mathbb{E}_{\hat{G}}\Big[(\|\nabla_{\hat{G}}\mathcal{D}(\hat{G})\|_2 - 1)^2\Big]}_{\text{gradient penalty}}, \tag{20}$$

where each expectation $\mathbb{E}[\cdot]$ is in practice the average over one batch. Consider the first two terms:

- $\mathbb{E}_{\tilde{G}\sim\mathcal{G}}[\mathcal{D}(\tilde{G})]$ is the average score the discriminator assigns to generated graphs. The discriminator minimises $\mathcal{L}_D$, so it pushes this term down, assigning *low* scores to generated graphs.
- $-\mathbb{E}_{G\sim p_{\text{data}}}[\mathcal{D}(G)]$ is the negative of the average score on real graphs. Minimising this term pushes the average score on real graphs *up*.

Together, these two terms encode one instruction: push real scores up, push generated scores down. The third term, the gradient penalty, is a stability device explained below.

3.4.4 Generator Loss

The generator's adversarial loss is the negation of the discriminator's "score on fake" term:

$$\mathcal{L}_{\text{GAN}} = -\mathbb{E}_{\tilde{G}\sim\mathcal{G}}[\mathcal{D}(\tilde{G})]. \tag{21}$$

The generator wants to maximise the score the discriminator assigns to its outputs. Since optimisers minimise by convention, we negate: minimising $-\mathbb{E}[\mathcal{D}(\tilde{G})]$ is equivalent to maximising $\mathbb{E}[\mathcal{D}(\tilde{G})]$.

Real graphs do not appear in $\mathcal{L}_{\text{GAN}}$. The generator never sees real data directly. It learns about the real distribution only through the gradient signal from $\mathcal{D}$: real graphs train $\mathcal{D}$, and $\mathcal{D}$'s gradient informs the generator how to adjust its outputs. This indirect chain, from real data to $\mathcal{D}$'s parameters to $\mathcal{D}$'s gradient to the generator's parameters, is the core mechanism of GANs.

3.4.5 Why WGAN-GP and Not a Vanilla GAN

In a vanilla GAN, the discriminator ends in a sigmoid, so its output is a value in $[0, 1]$ interpreted as the probability of the input being real. The problem is gradient saturation.

Once $\mathcal{D}$ can reliably separate real from generated graphs, the sigmoid output for generated graphs is pushed toward 0. The derivative of the sigmoid $\sigma(x) = 1/(1+e^{-x})$ is $\sigma(x)(1-\sigma(x))$, which approaches zero when $\sigma(x) \approx 0$ or $\sigma(x) \approx 1$. The generator's weight updates are computed via the chain rule as a product of derivatives, one of which is this sigmoid derivative. A near-zero factor makes the entire product near-zero, and the generator stops learning. This is the core difficulty of vanilla GAN training.

WGAN removes the sigmoid. The discriminator's output is an unbounded real number with no flat region. When $\mathcal{D}$ becomes strong, its output for generated graphs continues decreasing, and the derivative remains non-degenerate, so the generator continues to receive a gradient signal.

Removing the sigmoid introduces a new problem: without any bound on $\mathcal{D}$'s output, it can reduce its loss without limit by scaling its weights up. Doubling all weights roughly doubles the output gap between real and generated scores. The gradient penalty in Equation (20) prevents this. The term

$$\lambda_{\text{GP}}\, \mathbb{E}_{\hat{G}}\Big[(\|\nabla_{\hat{G}}\mathcal{D}(\hat{G})\|_2 - 1)^2\Big]$$

penalises deviation of the gradient magnitude of $\mathcal{D}$'s output (with respect to its input $\hat{G}$) from 1. Here $\hat{G} = \epsilon G + (1-\epsilon)\tilde{G}$ with $\epsilon \sim \text{Uniform}(0,1)$ is a random interpolation between a real and a generated graph, so the penalty is evaluated at points between the two distributions. The coefficient $\lambda_{\text{GP}} = 10$.

This constraint has two effects. First, $\mathcal{D}$ cannot reduce its loss by weight rescaling, because rescaling increases the gradient magnitude and the penalty dominates. Second, the gradient flowing from $\mathcal{D}$ into the generator remains at a bounded magnitude, neither vanishing (the vanilla GAN failure) nor exploding (the unconstrained WGAN failure).

In summary: a vanilla GAN's sigmoid saturates when $\mathcal{D}$ is confident, killing the generator's gradient. WGAN-GP removes the sigmoid and adds a gradient penalty that keeps the gradient magnitude near 1. The generator therefore receives a non-degenerate learning signal regardless of $\mathcal{D}$'s strength.

3.4.6 GNN Guidance Loss

The adversarial loss enforces structural realism but does not steer generation toward any particular class. The GNN guidance loss maximises the pretrained classifier's confidence on the target class c:

$$\mathcal{L}_{\text{GNN}} = -\log p_{\Phi}(y = c \mid \tilde{G}), \tag{22}$$

where $p_{\Phi}(y = c \mid \tilde{G})$ is the softmax probability that the pretrained k-GNN Φ assigns to class c for the generated graph $\tilde{G}$. This is the negative log-likelihood of the target class. The classifier's weights are frozen; only the generator's weights are updated. Minimising $\mathcal{L}_{\text{GNN}}$ pushes the generator to produce graphs that the classifier assigns to class c.

3.4.7 Dynamic Weighting

The balance parameter $\lambda(t)$ in Equation (19) follows a sigmoid schedule [Sun et al.(2025)]:

$$\lambda(t) = \lambda_{\min} + (\lambda_{\max} - \lambda_{\min}) \cdot \sigma\left(k \cdot \left(\frac{2(t/T - p)}{1-p} - 1\right)\right), \tag{23}$$

where T is the total number of training iterations ($T = 300$ epochs in our experiments), $p = 0.4$ is the fraction of training before λ begins increasing, $k = 10$ controls the steepness of the transition, and $\sigma(x) = 1/(1 + e^{-x})$ is the logistic sigmoid.

When $t \ll pT$ (the first $\sim$120 epochs), the sigmoid argument is strongly negative, giving $\lambda(t) \approx 0$: the total loss is $\mathcal{L}_G \approx \mathcal{L}_{\text{GAN}}$, and the generator focuses on producing structurally realistic graphs without class-specific pressure. As t passes pT, $\lambda(t)$ increases toward 1: the total loss shifts to $\mathcal{L}_G \approx \mathcal{L}_{\text{GNN}}$, directing the generator toward class-specific patterns. Without this schedule, the generator collapses to structurally degenerate graphs that satisfy the classifier but bear no resemblance to real data, since $\mathcal{L}_{\text{GNN}}$ can be minimised by any graph that triggers the correct class output, regardless of structural plausibility.

3.4.8 Structural Regularisation

The generator outputs a fixed-size $N \times N$ adjacency matrix and $N \times D$ feature matrix ($N = 28$ for MUTAG, $N = 50$ for PROTEINS). Without explicit constraints, the Gumbel-Softmax sampling activates most slots and settles on atom-type distributions that satisfy the classifier but do not resemble real class members. $\mathcal{L}_{\text{GAN}}$ constrains these statistics only indirectly through the discriminator, and the signal is too weak to prevent these failures. The regularisation term $\mathcal{L}_{\text{reg}}$ supplies direct supervision on two graph-level statistics: graph size and atom-type frequencies. Both targets are precomputed once from the real training set and remain fixed during training.

The regulariser decomposes as $\mathcal{L}_{\text{reg}} = \mathcal{L}_{\text{size}} + \mathcal{L}_{\text{type}}$, combined into the full expression:

$$\mathcal{L}_{\text{reg}} = \lambda_{\text{size}} \cdot \left(\frac{\hat{n} - \bar{n}_c}{\bar{n}_c} \right)^2 + \lambda_{\text{type}} \cdot \sum_{i=1}^{D} \left(\hat{p}_i - p_i^c \right)^2, \tag{24}$$

where $\mathcal{L}_{\text{size}}$ penalises deviation from the class mean graph size and $\mathcal{L}_{\text{type}}$ penalises deviation from the class atom-type distribution. We now derive each term.

Size penalty. The generator outputs a fixed $N \times N$ matrix with no flag marking which slots are active. A slot is effectively active if it has at least one edge to another slot, so the natural notion of graph size is the number of slots with at least one edge.

Define the soft degree of slot v as the row-sum of the soft adjacency matrix:

$$d_v = \sum_{u=1}^{N} \tilde{A}_{vu}.$$

A slot with no edges has $d_v \approx 0$; a slot with one edge has $d_v \approx 1$; a slot with three edges has $d_v \approx 3$. The naive active-slot count would use an indicator:

$$\hat{n}_{\text{naive}} = \sum_{v=1}^{N} \mathbb{1}[d_v > 0.5],$$

treating a slot as active if it has at least half an edge of connection. This does not work for training: the indicator is a step function with derivative zero almost everywhere and undefined at the step. Any gradient flowing from $\mathcal{L}_{\text{size}}$ through this path collapses to zero.

We therefore replace the step with a smooth approximation. The logistic sigmoid $\sigma(x) = 1/(1+e^{-x})$ approximates a step at $x = 0$. To place the transition at $d_v = 0.5$ and sharpen it, we shift the argument by 0.5 and scale by 10:

$$\hat{n} = \sum_{v=1}^{N} \sigma\big(10 \cdot (d_v - 0.5)\big).$$

The multiplier 10 is steep enough that slots with $d_v < 0.3$ contribute near zero and slots with $d_v > 0.7$ contribute near one:

d_v	0	0.3	0.5	0.7	1.0
$\sigma(10(d_v - 0.5))$	0.007	0.12	0.50	0.88	0.99

The expression approximates the active-slot count on the forward pass while providing non-zero gradients everywhere on the backward pass.

With $\hat{n}$ (soft count of active slots) and $\bar{n}_c$ (mean node count of real class-c graphs, pre-computed once), the size penalty is

$$\mathcal{L}_{\text{size}} = \lambda_{\text{size}} \left(\frac{\hat{n} - \bar{n}_c}{\bar{n}_c} \right)^2.$$

Two design choices matter. The squared error is smooth at zero with derivative vanishing as the error approaches zero, so the generator settles into the target without oscillation; the absolute value has a non-differentiable corner at zero and a constant-magnitude gradient. The division by $\bar{n}_c$ converts the error into a relative error, making the penalty scale-invariant across datasets: a 5-node error on MUTAG class 0 ($\bar{n}_c \approx 13.9$) is a $\sim$36% failure, while the same error on a dataset with $\bar{n}_c \approx 40$ is only $\sim$12.5%. Without this normalisation, λ_{size} would need to be retuned for every new dataset.

The coefficient $\lambda_{\text{size}} = 10$ is ten times larger than λ_{type} because a size violation breaks the graph structurally (a 50-node molecule where the class mean is 14 is unrecognisable), while atom-type violations are a softer failure.

Atom-type penalty. For each class c, the target distribution $p^c = (p_1^c, \ldots, p_D^c)$ is the frequency of each atom type in real class-c graphs, where p_i^c is the fraction of atoms of type i. This is a probability vector: entries in $[0, 1]$ summing to 1. For MUTAG class 0, $p^c \approx (0.85, 0.08, 0.04, \ldots)$ across the $D = 7$ atom types, with carbon dominating. Computed once, stored, reused.

The generated frequencies are computed from $\tilde{X}$. Each row of $\tilde{X}$ is nearly one-hot after Gumbel-Softmax, so summing column i counts how many slots selected atom type i:

$$\hat{p}_i = \frac{1}{N} \sum_{v=1}^{N} \tilde{X}_{v,i}.$$

Both $\hat{p}$ and p^c are probability distributions over the same D atom types. The penalty is the squared L2 distance between them, summed across types:

$$\mathcal{L}_{\text{type}} = \lambda_{\text{type}} \sum_{i=1}^{D} (\hat{p}_i - p_i^c)^2,$$

with $\lambda_{\text{type}} = 1$. The coefficient is smaller than λ_{size} because the sum already accumulates D squared errors and reaches meaningful magnitudes with a unit weight.

The natural distance between probability distributions is KL divergence, not L2. We use L2 for three reasons:

1. KL is undefined when $p_i^c = 0$, because the formula contains $\log(\hat{p}_i / p_i^c)$. On MUTAG, some classes contain zero instances of some atom types, so this is a concrete problem.
2. KL is asymmetric: $D_{\text{KL}}(\hat{p} \,\|\, p^c) \neq D_{\text{KL}}(p^c \,\|\, \hat{p})$. The two directions favour different failure modes and selecting one requires arbitrary justification. L2 is symmetric by construction.
3. KL has a gradient with a $1/p_i^c$ factor that explodes for rare atom types. The L2 gradient is $2(\hat{p}_i - p_i^c)$, bounded and stable.

Gradient path. $\mathcal{L}_{\text{reg}}$ has the shortest gradient path of the three loss components. Its gradients flow from scalar arithmetic on $(\tilde{A}, \tilde{X})$ through the Gumbel-Softmax backward pass into the generator MLP, without invoking the discriminator or the classifier. It is therefore the cheapest term per training step, since every other term requires a full network forward and backward pass while $\mathcal{L}_{\text{reg}}$ is closed-form. The target class enters only through the precomputed statistics $\bar{n}_c$ and p^c, so changing the target class requires no change to the computation graph.

3.5 Dense Wrapper for Differentiable k-GNN Inference

This section explains why the pretrained classifier cannot be used directly inside the generator's training loop, and how we solve this. The 1-GNN dense wrapper is taken from GIN-Graph [Sun et al.(2025)]; the same logic is applied to the 2-GNN branch of the 1-2-GNN classifier.

3.5.1 The Sparse Implementation

The classifier is trained on discrete, sparse graphs. A MUTAG molecule has around 20 edges out of $28 \times 28 = 784$ possible adjacency entries. Representing the graph as a full $N \times N$ matrix and computing $\mathbf{A}\,\mathbf{H}^{(t-1)}$ as a dense matrix product would waste computation on the zero entries.

The classifier instead uses PyTorch Geometric's sparse representation. A graph is stored as an `edge_index` tensor of shape $[2, |E|]$: a list of edges where the first row holds source indices and the second row holds destination indices. For a MUTAG molecule with 20 edges (40 directed entries for an undirected graph), this tensor has 40 columns instead of 784 matrix entries. Message passing is done via `scatter_add`: for each edge (i, j), the feature vector of node j is transformed by $\mathbf{W}_2^{(t)}$ and added to an accumulator for node i. After processing all edges, the accumulator for each node contains the sum of its neighbours' transformed features, which is the same result as $\mathbf{A}\,\mathbf{H}^{(t-1)}\,\mathbf{W}_2^{(t)}$, without materialising $\mathbf{A}$.

The classifier's forward function has the signature `forward(x, edge_index)`. It does not accept an adjacency matrix. For classifier training this is appropriate: the adjacency is a fixed discrete input, no gradients flow through it, and the sparse form is faster.

Two points matter for what follows: (i) the sparse form computes exactly the same function as the dense matrix form on discrete graphs, so it is an implementation choice, not a modelling choice; and (ii) the sparse form was chosen for speed and memory reasons that no longer apply inside the generator's training loop.

3.5.2 The Problem the Generator Creates

The generator outputs a continuous adjacency matrix $\tilde{A} \in [0,1]^{N \times N}$, where each entry is a soft edge weight produced by the Gumbel-Softmax straight-through estimator (Section 3.3). Training requires backpropagating from $\mathcal{L}_{\text{GNN}} = -\log p_\Phi(y = c \mid \tilde{A}, \tilde{X})$ (Equation 22) to the generator's weights. This requires the partial derivative $\partial f_\Phi / \partial \tilde{A}$, the gradient of the classifier's output with respect to its adjacency input.

The classifier's sparse forward function does not take an adjacency matrix directly. It takes an `edge_index`. Converting $\tilde{A}$ to an edge list requires thresholding: keep entries where $\tilde{A}_{ij} > 0.5$ and discard the rest. Thresholding is a step function, so its derivative is zero everywhere except at the discontinuity. After thresholding, the `edge_index` consists of integer indices with no autograd connection to the continuous values they came from. The partial derivative $\partial f_\Phi / \partial \tilde{A}$ therefore becomes zero, and the generator receives no learning signal from $\mathcal{L}_{\text{GNN}}$.

The failure is mechanical. The chain rule multiplies derivatives along the path from $\tilde{A}$ to $\mathcal{L}_{\text{GNN}}$. One of those factors is the derivative of the thresholding step, which is zero, so the entire product is zero and the generator's weights do not update.

3.5.3 The Fix

The fix is to rewrite the classifier's forward pass to consume $\tilde{A}$ directly as a continuous matrix. The mathematical function is unchanged: on a discrete $\{0,1\}$ adjacency matrix, the dense rewrite returns the same output as the sparse version. The implementation differs: where the sparse version uses `scatter_add` on an edge list, the dense version uses matrix multiplication $\tilde{A}\,\mathbf{H}^{(t-1)}$ via `torch.matmul`. Autograd can differentiate this operation entrywise, so every element of $\tilde{A}$ has a well-defined derivative and the chain rule no longer contains a zero threshold factor.

The pretrained weights $\mathbf{W}_1^{(t)}, \mathbf{W}_2^{(t)}, \mathbf{W}_3^{(\ell)}, \mathbf{W}_4^{(\ell)}$ are copied from the sparse model unchanged. These weights parameterise the update rule (how features are transformed and combined), not the implementation (how edges are iterated). They are valid in both code paths and remain frozen throughout GIN-Graph training; only the generator's weights are updated.

The dense wrapper is not a different model. It is the same model with a different forward function that accepts continuous adjacency matrices and maintains a gradient path from $\tilde{A}$ to the classifier's output.

3.5.4 Dense 1-GNN

For the 1-GNN branch, the rewrite is direct. The node-level update (Equation 5) translates to dense batched computation. For a batch of $B = 64$ graphs with node features

$\mathbf{X} \in \mathbb{R}^{B\times N\times d}$ and adjacency $\mathbf{A} \in [0,1]^{B\times N\times N}$:

$$\mathbf{H}^{(t)} = \sigma\Big(\mathbf{H}^{(t-1)}\mathbf{W}_1^{(t)} + \mathbf{A}\,\mathbf{H}^{(t-1)}\mathbf{W}_2^{(t)}\Big), \tag{25}$$

where the batch dimension is implicit. The product $\mathbf{A}\,\mathbf{H}^{(t-1)}$ replaces sparse neighbourhood aggregation. Each entry $\tilde{A}_{ij}$ participates directly in the multiplication, so autograd can differentiate with respect to it. The weights $\mathbf{W}_1^{(t)}$ and $\mathbf{W}_2^{(t)}$ are copied from the pretrained sparse model and kept frozen. This rewrite follows GIN-Graph [Sun et al.(2025)] and similar dense formulations for differentiable graph generation. It is stated here to contrast with the 2-GNN case.

3.5.5 Dense 2-GNN

The same sparse-to-dense logic is applied to the 2-GNN branch. The construction is more involved than for the 1-GNN because the 2-set neighbourhood depends on adjacency lookups, which must also be made continuous.

Recall from Section 3.2 that the 2-GNN operates on node pairs $\{i, j\}$, and the neighbourhood definition (Equation 9) requires checking whether a candidate node w is adjacent in the original graph: $A_{iw} = 1$ when replacing i, or $A_{jw} = 1$ when replacing j. In the sparse implementation, this adjacency check uses a CSR (compressed sparse row) lookup: given node i, iterate over its edge list to find all w with $(i, w) \in E$. This is fast for discrete graphs, but it is the same type of discrete operation that blocks gradients, since the neighbour set is a list of integer indices with no derivative connecting it to $\tilde{A}$.

In the dense rewrite, this binary check becomes a continuous weight. Instead of checking whether w is a neighbour of i (answer: 0 or 1), we use $\tilde{A}_{iw} \in [0, 1]$ directly. Candidate pair features $\mathbf{g}[j, w]$ are multiplied by $\tilde{A}_{iw}$, contributing to the aggregation in proportion to $\tilde{A}_{iw}$. When $\tilde{A}_{iw} \approx 0$, the contribution is zero; when $\tilde{A}_{iw} \approx 1$, it is full; intermediate values contribute proportionally. On discrete $\{0, 1\}$ inputs, this reproduces the original neighbourhood rule exactly.

First, we construct pair features from the 1-GNN node embeddings $\mathbf{H}^{(T_1)} \in \mathbb{R}^{B\times N\times d_T}$. For each pair (i, j), we concatenate the two node embeddings in canonical min–max order with the continuous adjacency value $\tilde{A}_{ij}$:

$$\mathbf{F}[i, j] = \big[\mathbf{h}_{\min(i,j)} \parallel \mathbf{h}_{\max(i,j)} \parallel A_{ij}\big] \in \mathbb{R}^{2d_T+1}, \tag{26}$$

The canonical ordering $(\min, \max)$ ensures $\mathbf{F}[i, j] = \mathbf{F}[j, i]$, matching the unordered pair semantics of Equation (8). The last entry A_{ij} is the isomorphism type indicator. In the sparse model this was a hard bit $\mathbb{1}[(i, j) \in E] \in \{0, 1\}$; here it is the continuous value from the generator.

For pair-level message passing, recall from Equation (9) that each 2-set $\{i, j\}$ has two kinds of neighbours: pairs formed by replacing i with some w adjacent to i, and pairs formed by replacing j with some w adjacent to j. In the dense version, "adjacent to i" is the continuous weight $\tilde{A}_{iw}$. Using the transformed features $\mathbf{g} = \mathbf{F}\,\mathbf{W}_4^{(\ell)} \in \mathbb{R}^{B\times N\times N\times d_T}$,

the aggregation for pair (i, j) sums over all candidate w, weighted by $\tilde{A}$:

$$\text{agg}_1[i,j] = \sum_{\substack{w=1 \\ w \neq j}}^{N} A_{iw} \cdot \mathbf{g}[j,w] \qquad \text{(replace } i \text{ with } w \text{ adjacent to } i\text{)}, \tag{27}$$

$$\text{agg}_2[i,j] = \sum_{\substack{w=1 \\ w \neq i}}^{N} A_{jw} \cdot \mathbf{g}[i,w] \qquad \text{(replace } j \text{ with } w \text{ adjacent to } j\text{)}. \tag{28}$$

The exclusions $w \neq j$ in agg_1 and $w \neq i$ in agg_2 prevent including the self-pair $\{j, j\}$ or $\{i, i\}$, which is not a valid 2-set. The self-loop exclusion ($A_{ii} = 0$) removes $w = i$ from agg_1 and $w = j$ from agg_2, but the terms $w = j$ in agg_1 and $w = i$ in agg_2 must be explicitly masked. We zero the diagonal of $\mathbf{g}$ (setting $\mathbf{g}[k,k] = \mathbf{0}$ for all k) before summation, which eliminates both boundary terms: $\mathbf{g}[j,j] = \mathbf{0}$ removes the $w = j$ term in agg_1, and $\mathbf{g}[i,i] = \mathbf{0}$ removes the $w = i$ term in agg_2.

These sums are computed via Einstein summation (`einsum`) over the masked $\mathbf{g}$. The full 2-GNN layer update is:

$$\mathbf{F}^{(\ell+1)} = \sigma\Big(\mathbf{F}^{(\ell)}\,\mathbf{W}_3^{(\ell)} + \text{agg}_1^{(\ell)} + \text{agg}_2^{(\ell)}\Big). \tag{29}$$

After $T_2 = 2$ layers, the 2-GNN readout sums over the upper-triangular entries (one entry per canonical pair):

$$\mathbf{h}_G^{(2)} = \sum_{i<j} \mathbf{F}^{(T_2)}[i,j]. \tag{30}$$

3.5.6 Summary

The dense wrapper preserves the exact mathematical behaviour of the pretrained classifier on discrete graphs while extending its domain to continuous adjacency matrices. It is an implementation change, not a model change: all pretrained weights are reused unchanged. For the 1-GNN branch, the change is replacing `scatter_add` with `matmul`, as in GIN-Graph [Sun et al.(2025)]. For the 2-GNN branch, the discrete adjacency check $A_{iw} \in \{0, 1\}$ is replaced by a continuous weight $\tilde{A}_{iw} \in [0, 1]$ that participates in the computation graph. Without this, the GIN-Graph training loop, which requires $\partial f_\Phi / \partial \tilde{A}$ to be well-defined, cannot be run against a 1-2-GNN classifier.

3.6 Validation Score

At evaluation time, both the generator and the classifier are frozen; no gradients are computed. We sample 100 graphs per class at temperature $\tau = 0.1$ (which produces sharper, more discrete outputs than the training temperature of $\tau = 1.0$) and assign each graph a score that quantifies its quality as an explanation. The score must be low if the graph fails on any of three axes (class identity, internal representation, or structural plausibility), so that success on one axis cannot compensate for failure on another.

The composite validation score [Sun et al.(2025)] is:

$$v = (s \cdot p \cdot d)^{1/3}, \tag{31}$$

where p measures the classifier's prediction, s measures where the graph sits in the classifier's internal embedding space, and d measures whether the graph has a realistic amount of connectivity.

The geometric mean enforces non-compensation between components. If a generated graph has $p = 0.95$, $s = 0.95$, but $d = 0.05$ (correct class, correct embedding region, wrong connectivity), the geometric mean gives $v \approx 0.36$, correctly indicating failure. An arithmetic mean would give $v \approx 0.65$, above the validity threshold, hiding the structural problem. We now define each component.

3.6.1 Prediction Probability (p)

The prediction probability is the frozen classifier's softmax output for the target class:

$$p = p_\Phi(y = c \mid \tilde{G}).$$

This is the same quantity the generator was trained to maximise via $\mathcal{L}_{\text{GNN}}$ (Equation 22). For a well-trained generator, p is close to 1 at evaluation time. This is expected: the generator spent hundreds of epochs optimising this quantity. The component p therefore functions as a sanity check that the generator converged, not as a discriminator between good and poor generators. A generator producing graphs with $p < 0.5$ did not converge.

3.6.2 Embedding Similarity (s)

For any graph G the classifier processes, the forward pass produces a single vector $\mathbf{h}_G$ representing the entire graph. In the 1-GNN, this vector is the sum pool of the final node embeddings: $\mathbf{h}_G = \sum_{v \in V} \mathbf{h}_v^{(T_1)} \in \mathbb{R}^{d_T}$ (Equation 6). In the 1-2-GNN, it is the concatenation of the node-level and pair-level readouts: $\mathbf{h}_G = [\mathbf{h}_G^{(1)} \| \mathbf{h}_G^{(2)}] \in \mathbb{R}^{2d_T}$ (Equation 11). One vector per graph, in the classifier's embedding space. Every forward pass on any graph produces exactly one such vector.

The *class centroid* $\boldsymbol{\mu}_c$ is the mean of $\mathbf{h}_G$ over all real training graphs in class c:

$$\boldsymbol{\mu}_c = \frac{1}{|\mathcal{G}_c|} \sum_{G \in \mathcal{G}_c} \mathbf{h}_G.$$

For MUTAG class 0, $\mathcal{G}_c$ contains 125 molecules, so $\boldsymbol{\mu}_c$ is the average of 125 embedding vectors. The centroid sits in the same space as any individual $\mathbf{h}_G$ and marks the average location where real class-c graphs lie according to the classifier. It is computed once using the classifier's sparse implementation (real graphs are discrete, so sparse is appropriate), stored in the GIN-Graph checkpoint, and reused for every evaluation.

The embedding similarity is then the cosine similarity between the generated graph's embedding and this centroid:

$$s = \cos(\mathbf{h}_{\tilde{G}}, \boldsymbol{\mu}_c) = \frac{\mathbf{h}_{\tilde{G}} \cdot \boldsymbol{\mu}_c}{\|\mathbf{h}_{\tilde{G}}\| \, \|\boldsymbol{\mu}_c\|}, \quad \text{where } \boldsymbol{\mu}_c = \frac{1}{|\mathcal{G}_c|} \sum_{G \in \mathcal{G}_c} \mathbf{h}_G. \tag{32}$$

Cosine similarity measures the angle between two vectors, normalised so that identical direction gives $s = 1$ and orthogonal vectors give $s = 0$. A high s means the generated

graph occupies the same region of embedding space as real class-c graphs. A low s means the classifier assigns the graph to class c but via an internal representation far from typical class members; the generator has found an input that triggers the correct output without resembling the real data in the classifier's internal representation.

Note on the computation of s. The GIN-Graph reference implementation [Sun et al.(2025)] uses prediction probability p as a proxy for embedding similarity, so it sets $s = p$ and no longer measures embedding alignment directly. In this work, s is the cosine similarity between the generated-graph embedding and the real class centroid. On PROTEINS Enzyme, that distinction matters. Both models achieve high p (0.919, 0.998) but low s (0.356, 0.151).

3.6.3 Degree Score (d)

The average degree of a graph is $\bar{d}_{\tilde{G}} = 2|\tilde{E}|/|\tilde{V}|$: total degree divided by node count, since each edge contributes 1 to each of its two endpoints. Let μ_d and σ_d be the mean and standard deviation of average degrees across real class-c graphs, both precomputed once from the training set. The degree score is:

$$d = \exp\left(-\frac{(\bar{d}_{\tilde{G}} - \mu_d)^2}{2\sigma_d^2}\right), \tag{33}$$

which is a Gaussian centred at the class mean μ_d with width σ_d. When $\bar{d}_{\tilde{G}} = \mu_d$, the score is $d = 1$. As the generated degree deviates from the mean, d decreases: one standard deviation off gives $d \approx 0.61$, two standard deviations give $d \approx 0.14$, three give $d \approx 0.01$. Classes with high variance in graph sizes get a wider (more permissive) Gaussian than classes with uniform sizes.

The degree score provides a structural sanity check that s and p cannot provide. Both s and p operate in the classifier's learned embedding space. A generator could produce graphs with correct class identity and correct embedding location but with edge counts far from any real graph. The degree score catches this failure mode. It does not capture fine-grained structural properties (motif frequencies, degree distributions), but it reliably rejects graphs with wrong connectivity at the coarsest level.

3.6.4 Validity Threshold

A generated graph is considered **valid** if two conditions hold: (i) $v > 0.5$, and (ii) $\bar{d}_{\tilde{G}}$ falls within 3 standard deviations of μ_d. The degree score d therefore appears twice: once as a soft component inside v, and once as a hard gate via the 3σ condition. The hard gate catches graphs with abnormal connectivity regardless of their s and p values. A graph with $\bar{d}_{\tilde{G}}$ far outside the class range is rejected even if it scores well on the other two components. The validity rates reported in Section 5 are filtered on both conditions.

3.6.5 Granularity (κ)

We additionally report:

$$\kappa = 1 - \min\left(1,\ \frac{n_{\tilde{G}}}{\bar{n}_c}\right), \tag{34}$$

where $n_{\tilde{G}}$ is the number of active (non-isolated) nodes in the generated graph and $\bar{n}_c$ is the mean node count of real class-c graphs (the same $\bar{n}_c$ used in $\mathcal{L}_{\text{size}}$, Equation 24). When $n_{\tilde{G}} \geq \bar{n}_c$, the ratio saturates at 1 and $\kappa = 0$: the explanation is graph-sized. When $n_{\tilde{G}} \ll \bar{n}_c$, κ approaches 1: the explanation is a small substructure.

In these experiments, $\kappa \approx 0$ for all configurations. This is a structural property of GIN-Graph, not a failure. The generator outputs fixed-size $N \times N$ tensors and $\mathcal{L}_{\text{size}}$ pushes $\hat{n}$ toward $\bar{n}_c$, so explanations are graph-sized by construction. Model-level explainers answer "what does a typical class member look like?" rather than "which subgraph drives this prediction?"; the latter requires instance-level methods such as GNNExplainer. We report κ for completeness and as a diagnostic; this limitation is discussed further in Section 6.

3.6.6 Evaluation Protocol

Each configuration generates 100 graphs and reports: (i) the validity rate, the fraction passing both the $v > 0.5$ and the 3σ degree conditions; (ii) the mean validation score, averaged across all 100 graphs; (iii) the component decomposition s, p, d (Table 6), where patterns such as high p with low s on PROTEINS Enzyme would not appear under the $s = p$ proxy; and (iv) the top-ranked explanations sorted by v, shown in the figures. All four quantities are forward-pass computations on frozen networks. No training or gradients are involved.

4 Experimental Setup

4.1 Datasets

We evaluate on two standard graph classification benchmarks (Table 1).

Table 1: Dataset statistics. GIN Gen is the maximum number of nodes used for explanation generation.

Dataset	Graphs	Node Feats	Max Nodes	GIN Gen	Task
MUTAG	188	7 (atom types)	28	28	Mutagenicity
PROTEINS	1113	3 (sec. str.)	620	50	Enzyme / non-enzyme

MUTAG [Debnath et al.(1991)] contains 188 molecular graphs labelled by mutagenic effect on *Salmonella typhimurium*. Nodes represent atoms (C, N, O, F, I, Cl, Br) and edges represent chemical bonds. The two classes are *Mutagen* (class 0, 125 graphs) and *Non-Mutagen* (class 1, 63 graphs).

PROTEINS [Borgwardt et al.(2005)] contains 1113 protein graphs where nodes are secondary structure elements (helix, sheet, coil/turn) connected if they are neighbours in the amino acid sequence or within a distance threshold in 3D space. The classes are *Non-Enzyme* (class 0) and *Enzyme* (class 1). For GIN-Graph generation, we cap the graph size at 50 nodes (the median protein graph has $\sim$26 nodes), as the explanations highlight key substructures rather than reproducing entire large graphs.

4.2 Model Configuration

All k-GNN models use a hidden dimension of $d_T = 64$. The 1-GNN uses $T_1 = 3$ message-passing layers; the 1-2-GNN uses 3 layers for the 1-GNN component and $T_2 = 2$ layers for the 2-GNN component. Both use dropout of 0.5 and are trained with Adam (lr = 0.01) for 100 epochs with cross-entropy loss. Data is split 80/20 into train/test with a fixed seed of 42.

The GIN-Graph generator uses a latent dimension $d_z = 32$, hidden dimension $d_h = 128$, and is trained for 300 epochs with Adam (lr = 0.001) and batch size 64. WGAN-GP uses $\lambda_{\text{GP}} = 10$ and trains the discriminator once per generator update ($n_{\text{critic}} = 1$). The dynamic weighting uses $p = 0.4$, $k = 10$, $\lambda_{\min} = 0$, $\lambda_{\max} = 1$.

Computational cost. Table 2 reports wall-clock times on CPU. The 2-GNN component incurs substantial overhead due to the $O(n^2)$ pair construction and message passing. On PROTEINS, the 1-2-GNN is $\sim$326$\times$ slower per forward pass than the 1-GNN, reflecting the cost of processing all $\binom{n}{2}$ node pairs for graphs with up to 620 nodes. GIN-Graph generation uses the dense wrapper, where the 1-2-GNN overhead is $\sim$36–74$\times$ at the fixed generation size ($N = 28$ for MUTAG, $N = 50$ for PROTEINS).

Table 2: Wall-clock inference times on CPU. k-GNN: sparse forward pass (batch of 64 graphs). GIN-Graph: dense wrapper generation (100 graphs).

Dataset	Model	k-GNN (ms/batch)	GIN-Graph gen (s/100)
MUTAG	1-GNN	1.9	0.02
MUTAG	1-2-GNN	99.5 (52×)	1.48 (74×)
PROTEINS	1-GNN	6.8	0.04
PROTEINS	1-2-GNN	2214 (326×)	1.43 (36×)

4.3 Evaluation Protocol

For each model-dataset-class combination, we generate 100 explanation graphs at evaluation temperature $\tau = 0.1$ and evaluate them using the validation score (Equation 31). We report:

- **Validity rate**: fraction of explanations passing degree and score thresholds;
- **Mean validation score**: averaged over all 100 generated graphs;
- **Component scores**: embedding similarity (s), prediction probability (p), degree score (d);
- **Granularity**: how fine-grained the explanations are.

5 Results

5.1 k-GNN Classification Performance

Table 3 summarises the classification results. On MUTAG, both models achieve identical best test accuracy of 89.5%, indicating that the additional pairwise message passing does

not provide a classification advantage on this small molecular dataset. On PROTEINS, both models achieve comparable classification accuracy (79.8% vs. 78.9%) after 100 epochs of training, with the 1-2-GNN now processing all $\binom{n}{2}$ node pairs via a numba-accelerated kernel instead of sampling a subset. The different best epochs on PROTEINS (120 for 1-GNN, 75 for 1-2-GNN) suggest different training dynamics once full pairwise structure is included, but under a single split this should be read as descriptive rather than conclusive.

Table 3: k-GNN classification results. Best test accuracy reported over all epochs.

Dataset	**Model**	**Params**	**Final Test Acc**	**Best Acc**	**Best Epoch**
MUTAG	1-GNN	21,570	81.6%	89.5%	84
MUTAG	1-2-GNN	50,370	76.3%	89.5%	18
PROTEINS	1-GNN	21,058	75.8%	79.8%	120
PROTEINS	1-2-GNN	49,858	73.1%	78.9%	75

5.2 GIN-Graph Explanation Quality: MUTAG

Table 4 presents the GIN-Graph results on MUTAG. All configurations were fully trained for 300 epochs, enabling fair cross-model comparison on both classes.

Table 4: GIN-Graph explanation quality on MUTAG. All models trained for 300 epochs.

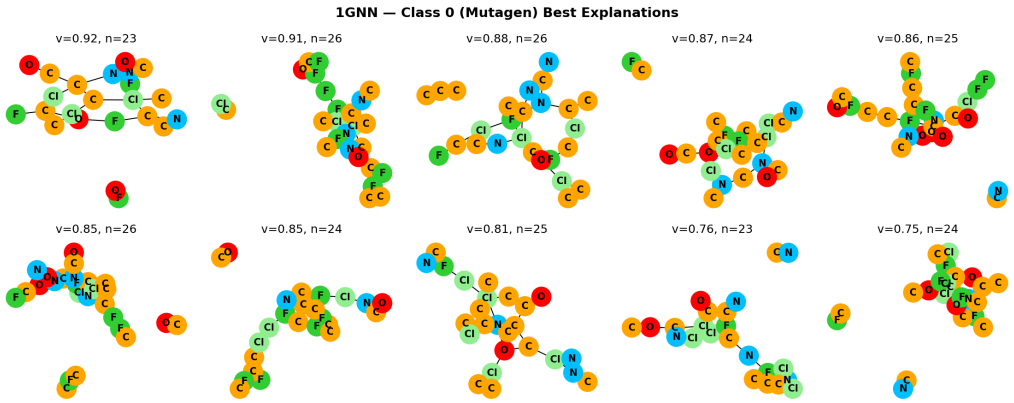
Class	**Model**	**Valid%**	**Val Score**	s	p	d
0 (Mutagen)	1-GNN	36%	0.373	0.701	1.000	0.260
0 (Mutagen)	1-2-GNN	**41%**	**0.434**	**0.771**	1.000	**0.314**
1 (Non-Mutagen)	1-GNN	78%	0.681	**0.935**	1.000	0.490
1 (Non-Mutagen)	1-2-GNN	**82%**	**0.735**	0.933	1.000	**0.565**

Both models achieve successful generation on both classes, with the 1-2-GNN showing somewhat higher validity rates (82% vs. 78% on class 1, 41% vs. 36% on class 0) and validation scores (0.735 vs. 0.681 on class 1). However, under the single-seed setup these quantitative gaps should be treated cautiously (Section 7). The more important result is the *qualitative divergence* in what the two models generate.

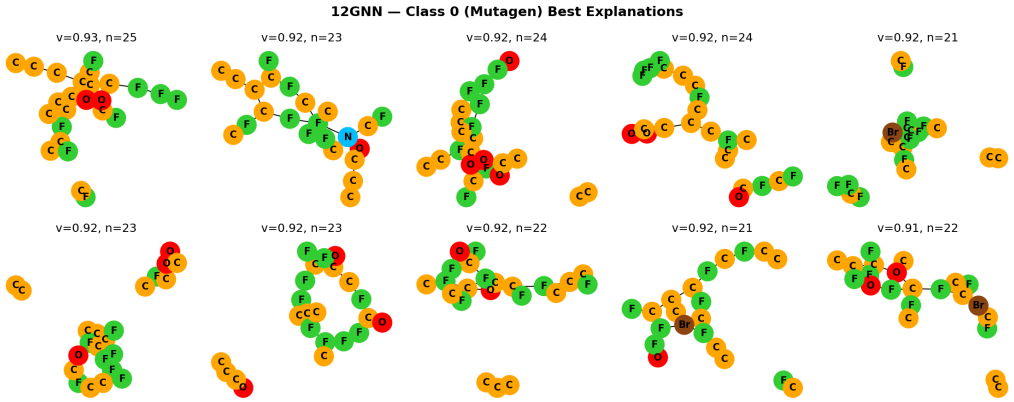
Despite identical classification accuracy (89.5%), the two models produce strikingly different class-0 explanation families (Figure 1). Across the validation-ranked top explanation set, the 1-GNN remains C/N/O-dominated (42.9% C, 13.9% N, 13.2% O), while the 1-2-GNN shifts toward a much more halogen-heavy pattern (31.1% F, 1.1% Br) with almost no nitrogen (0.4% N). Because these percentages come from the top-ranked set rather than the full generated population, they should be read as representative high-scoring prototypes. The population-level comparison in Section 5.6 shows the same directional pattern: on MUTAG class 0, the 1-2-GNN generator overproduces halogens much more strongly than the 1-GNN.

Class 0 remains the harder class for both models, but this should not be read only as a failure case. MUTAG mutagens are structurally diverse, with nitro/amino-rich and halogen-rich signatures both appearing in the data. The contrast between the 1-GNN's

N/O-rich prototypes and the 1-2-GNN's halogen-rich prototypes therefore looks less like contradictory noise and more like evidence that the two architectures settle on different mutagenic signature families within a heterogeneous class. Both models still achieve perfect prediction probability ($p = 1.0$).



(a) 1-GNN explanations for Mutagen



(b) 1-2-GNN explanations for Mutagen

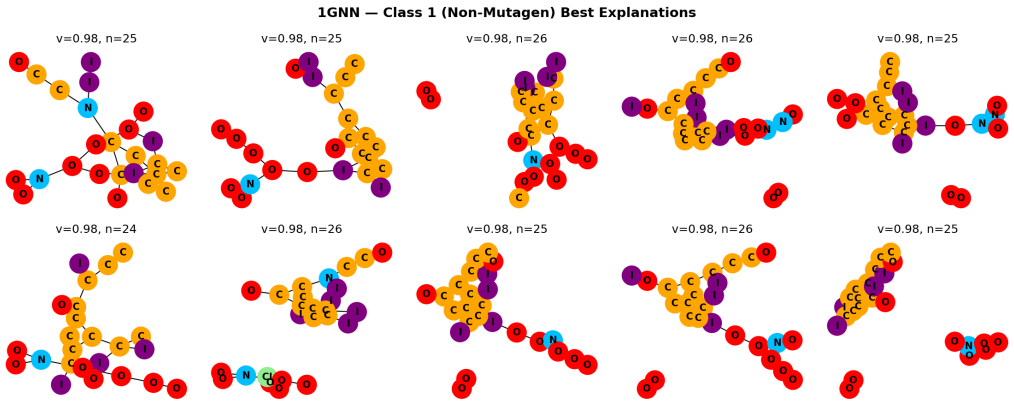
Figure 1: Top-ranked GIN-Graph explanations on MUTAG class 0 (Mutagen). The 1-GNN repeatedly produces mixed C/N/O scaffolds with scattered halogens, whereas the 1-2-GNN collapses onto narrower fluorine- and bromine-heavy prototype families. Node colours indicate atom types (see Figure 3).

Figure 2 shows the top-ranked explanation graphs for both models on MUTAG class 1.

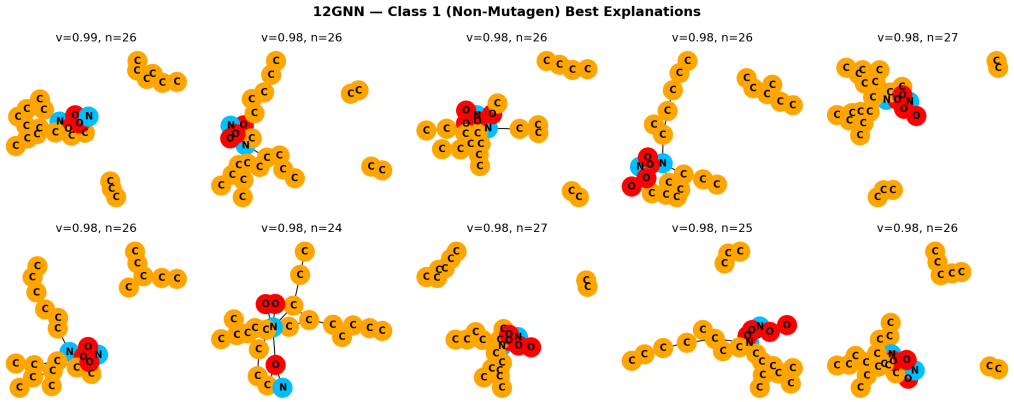
Both model families achieve near-perfect top validation scores (0.984–0.986), but the representative class-1 structures differ noticeably. The 1-GNN frequently produces longer oxygen- and iodine-bearing branches attached to a carbon scaffold, whereas the 1-2-GNN concentrates more strongly on compact carbon-dominated cores with small N/O motifs. As in class 0, these top-set percentages are conditioned on validation ranking; Table 8 in Section 5.6 provides the broader population-level composition.

5.3 GIN-Graph Explanation Quality: PROTEINS

Table 5 presents results on PROTEINS. All configurations were fully trained for 300 epochs with graph size and node type regularisation (see Section 3.4). Unlike MUTAG,



(a) 1-GNN explanations for Non-Mutagen



(b) 1-2-GNN explanations for Non-Mutagen

Figure 2: Top-ranked GIN-Graph explanations on MUTAG class 1 (Non-Mutagen). The 1-GNN explanations often attach oxygen- and iodine-bearing branches to a carbon backbone, while the 1-2-GNN explanations are more compact and carbon-dominated with smaller N/O motifs. Node colours indicate atom types (see Figure 3).

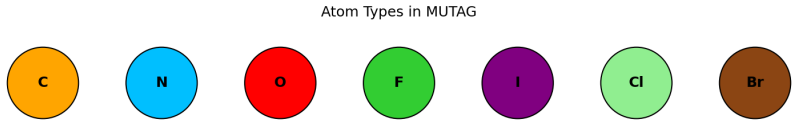


Figure 3: MUTAG atom type colour legend. Atoms: C (orange), N (blue), O (red), F (green), I (purple), Cl (light green), Br (brown).

the two models show complementary strengths, with each achieving higher scores on a different class.

Table 5: GIN-Graph explanation quality on PROTEINS. All models trained for 300 epochs with structural regularisation.

Class	**Model**	**Valid%**	**Val Score**	s	p	d
0 (Non-Enzyme)	1-GNN	100%	0.986	0.987	0.986	0.984
0 (Non-Enzyme)	1-2-GNN	100%	0.760	0.782	0.564	0.996
1 (Enzyme)	1-GNN	37%	0.330	0.356	0.919	0.837
1 (Enzyme)	1-2-GNN	94%	0.524	0.151	0.998	0.961

The 1-GNN achieves near-perfect scores on class 0 (Non-Enzyme) across all components ($v = 0.986$, $s = 0.987$, $p = 0.986$), while the 1-2-GNN achieves lower but still functional prediction probability ($p = 0.564$) and overall validation score ($v = 0.760$). On class 1 (Enzyme), both models achieve high prediction probability ($p = 0.919$ and 0.998), but the 1-GNN struggles with validity (37%) while the 1-2-GNN reaches 94% validity. Notably, both models show low embedding similarity on Enzyme ($s = 0.356$ and 0.151), indicating that correct classification alone does not imply close alignment with the real Enzyme centroid.

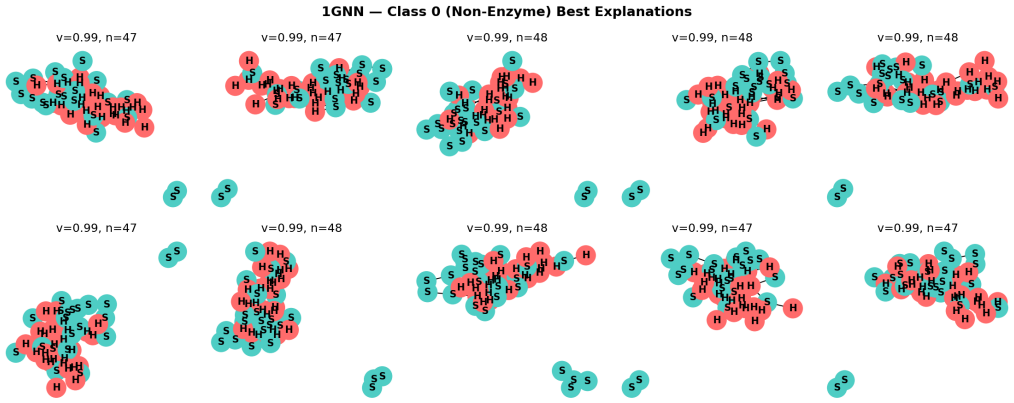
Both models achieve comparable classification accuracy (79.8% vs. 78.9%), so this complementary pattern reflects genuine differences in what the two architectures learn about protein structure rather than a simple classifier quality gap. The result is mixed rather than one-sided: the 1-GNN is strongest on Non-Enzyme by the original validation score, while the 1-2-GNN is strongest on Enzyme validity and later proves structurally closer on the population-level Non-Enzyme node-type distribution.

The representative examples make the class split concrete. The 1-GNN Non-Enzyme explanations are consistently large and compositionally balanced, matching the near-perfect class-0 metrics, whereas its Enzyme outputs are much less stable and include both compact plausible motifs and looser elongated structures, consistent with the 37% validity rate. The 1-2-GNN shows the opposite profile: its Non-Enzyme explanations are extremely regular, while its Enzyme explanations repeatedly collapse onto sheet-heavy cores. Taken together, the figures support a model-specific prototype story rather than a universal winner.

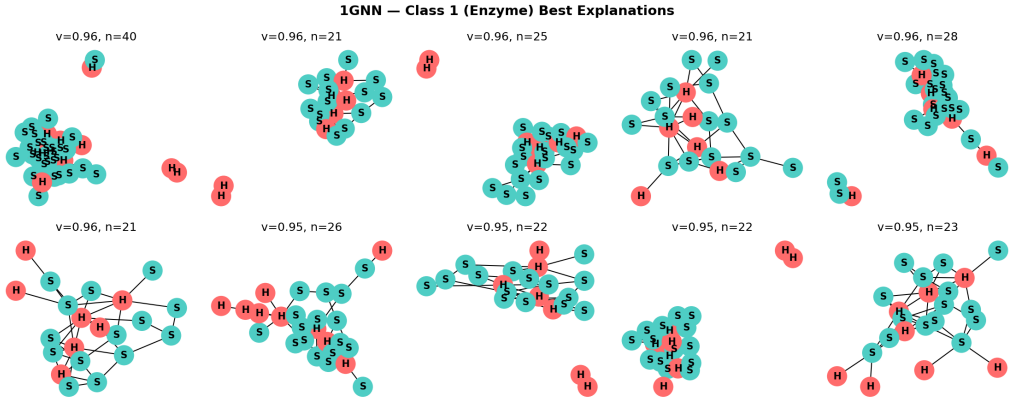
5.4 Metric Decomposition

Table 6 provides a detailed comparison of the three validation score components across all configurations, now with all models trained for 300 epochs.

On MUTAG, both models achieve perfect prediction probability ($p = 1.0$) on both classes, indicating that both classifiers provide strong gradient signal to the generators. The degree score shows the largest variation: the 1-2-GNN produces graphs with more realistic average degree on both classes (0.565 vs. 0.490 on class 1, 0.314 vs. 0.260 on class 0). Embedding similarity is comparable on class 1 ($s \approx 0.93$) and somewhat higher for the 1-2-GNN on class 0 (0.771 vs. 0.701).



(a) 1-GNN explanations for Non-Enzyme

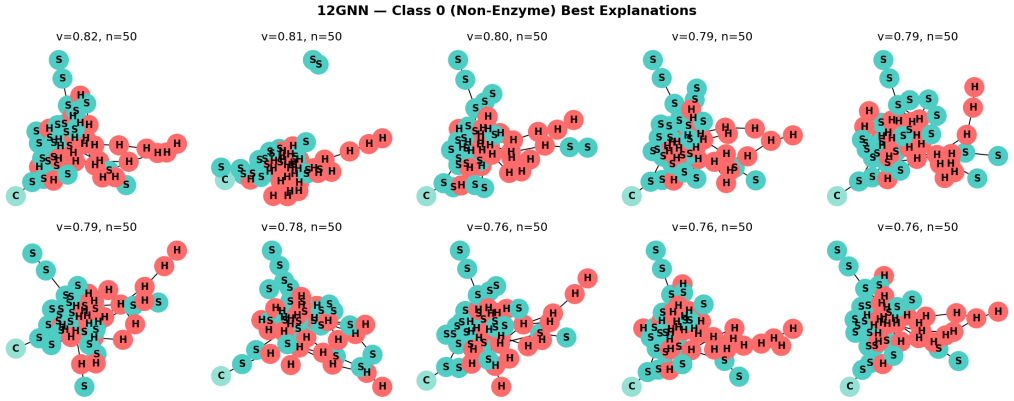


(b) 1-GNN explanations for Enzyme

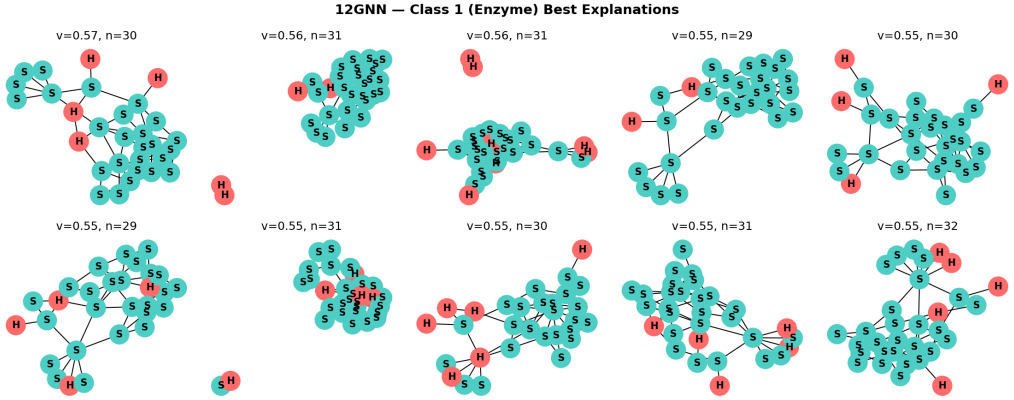
Figure 4: Top-ranked GIN-Graph explanations on PROTEINS (1-GNN). The Non-Enzyme explanations are large balanced H/S structures near the 50-node cap, while the Enzyme explanations are more variable in size and shape. Node colours indicate secondary structure types: helix (H, pink), sheet (S, teal), coil/turn (C, green).

Table 6: Validation score decomposition for all GIN-Graph models (300 epochs).

Dataset	**Class**	**Model**	s (emb. sim)	p (pred. prob)	d (degree)
MUTAG	0 (Mutagen)	1-GNN	0.701	1.000	0.260
MUTAG	0 (Mutagen)	1-2-GNN	0.771	1.000	0.314
MUTAG	1 (Non-Mut.)	1-GNN	0.935	1.000	0.490
MUTAG	1 (Non-Mut.)	1-2-GNN	0.933	1.000	0.565
PROTEINS	0 (Non-Enz.)	1-GNN	0.987	0.986	0.984
PROTEINS	0 (Non-Enz.)	1-2-GNN	0.782	0.564	0.996
PROTEINS	1 (Enzyme)	1-GNN	0.356	0.919	0.837
PROTEINS	1 (Enzyme)	1-2-GNN	0.151	0.998	0.961



(a) 1-2-GNN explanations for Non-Enzyme



(b) 1-2-GNN explanations for Enzyme

Figure 5: Top-ranked GIN-Graph explanations on PROTEINS (1-2-GNN). The Non-Enzyme explanations are highly regular 50-node H/S structures, whereas the Enzyme explanations repeatedly collapse onto sheet-dominated cores with smaller helix appendages. Class 0 explanations achieve validation scores $v = 0.76$–0.82 with 50 nodes, while class 1 explanations achieve $v = 0.52$–0.58 with 29–32 nodes.

On PROTEINS, the metric profiles reveal how each architecture interacts differently with the generator. The 1-GNN achieves near-perfect scores on class 0 ($s = 0.987$, $p = 0.986$, $d = 0.984$) but lower embedding similarity on class 1 ($s = 0.356$), indicating generated Enzyme graphs diverge from the real embedding centroid despite being correctly classified ($p = 0.919$). The 1-2-GNN achieves moderate prediction probability on class 0 ($p = 0.564$) with high embedding similarity ($s = 0.782$) and near-perfect degree score ($d = 0.996$), while on class 1 it achieves near-perfect prediction probability ($p = 0.998$) but very low embedding similarity ($s = 0.151$). The degree scores are generally high across PROTEINS ($d > 0.83$), confirming that structural realism is maintained by the WGAN-GP component.

5.5 Training Dynamics

Figure 6 shows a representative training curve from the MUTAG 1-2-GNN class 1 experiment. The three loss curves illustrate the interplay between the WGAN-GP adversarial training and the GNN guidance objective over 300 epochs.

During the first $\sim$120 epochs ($\lambda \approx 0$), the generator loss and discriminator loss dominate: the generator learns to produce structurally realistic graphs that fool the discriminator, without pressure to target a specific class. As the dynamic weighting $\lambda(t)$ increases (Equation 23), the GNN guidance loss takes over, steering the generator toward class-specific patterns. This transition is visible as the GNN loss drops sharply while the adversarial losses plateau.

The training behaviour varies across configurations. On MUTAG, all four generators converge smoothly, consistent with the small graph sizes ($N = 28$) and low feature dimensionality (7 atom types). On PROTEINS, the training dynamics are more complex: the 1-2-GNN class 0 generator achieves moderate prediction probability ($p = 0.564$ at evaluation), lower than the 1-GNN's near-perfect alignment ($p = 0.986$) on the same class. This gap reflects the difficulty of optimising through the 1-2-GNN's higher-dimensional embedding space ($\mathbb{R}^{128}$ vs. $\mathbb{R}^{64}$), where the generator must satisfy both node-level and pair-level classifier components simultaneously.

5.6 Generated Dataset Comparison

To evaluate the fidelity and cross-model transferability of generated explanations at scale, we generate 500 graphs per class from each GIN-Graph generator and compare the resulting synthetic datasets against the real data. This population-level analysis complements the per-graph metrics reported in Sections 5.2–5.4 by revealing systematic differences between what each model has learned. No retraining is performed. All graphs are generated from the existing checkpoints.

5.6.1 Structural Fidelity

Table 7 summarises the structural comparison. Across both datasets, degree distributions are matched more reliably than graph-size distributions. Kolmogorov–Smirnov statistics on degree range from 0.09 to 0.21, with generated mean degrees remaining close to the real values in every configuration. By contrast, the size KS statistics are substantially

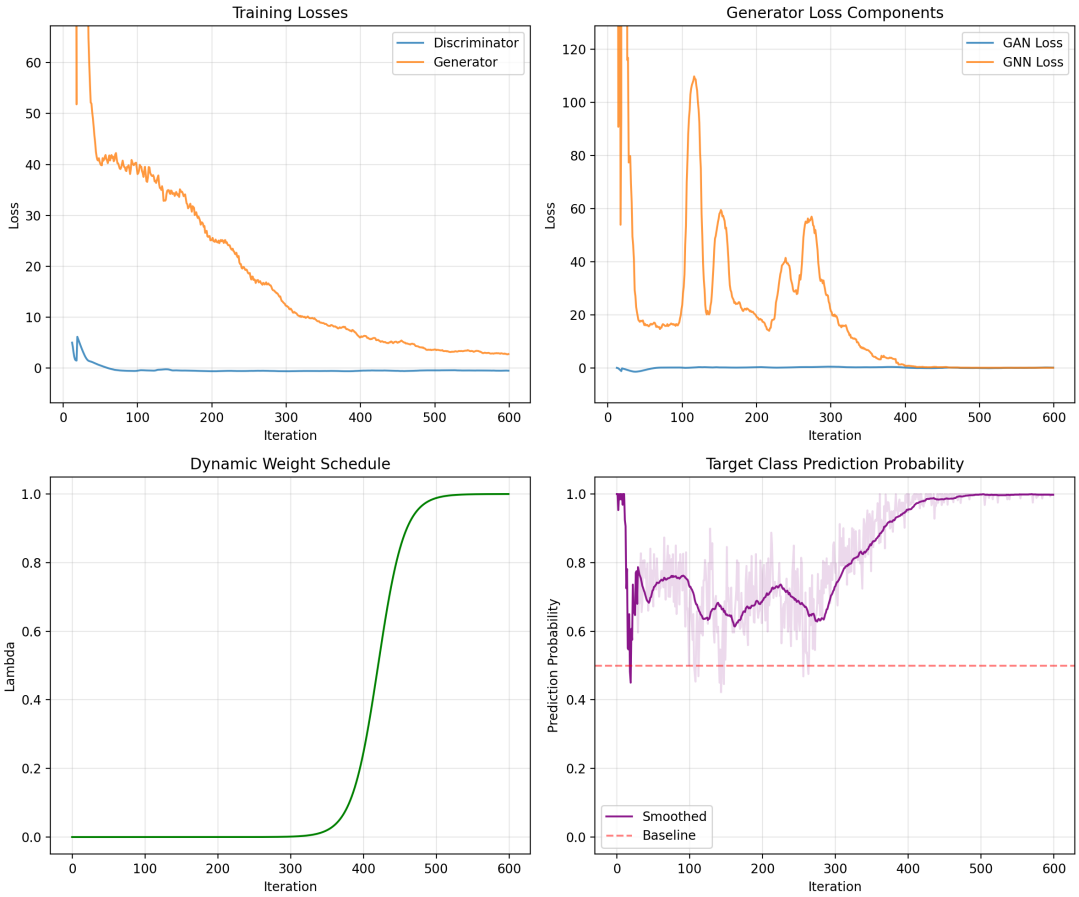


Figure 6: Training curves for GIN-Graph on MUTAG 1-2-GNN class 1, showing generator loss, discriminator loss, and GNN guidance loss over 300 epochs. The dynamic weighting schedule shifts emphasis from adversarial realism (early epochs) to class-specific GNN guidance (later epochs).

larger, indicating that the generators preserve local connectivity more successfully than the global population shape.

Size regularisation helps on PROTEINS, but only partially. On class 0 (Non-Enzyme), generated graphs average 46.6 nodes (1-GNN) and 50.0 nodes (1-2-GNN) versus a real mean of 48.7. On class 1 (Enzyme), the 1-GNN matches the real mean size closely (24.0 vs. 23.7), whereas the 1-2-GNN remains oversized (29.5 vs. 23.7). On MUTAG ($N = 28$), all generators still produce oversized graphs: the generated means lie between 23 and 27 nodes, while the real means lie between 14 and 20. This reflects the fixed generator size and the tendency of the Gumbel-Softmax sampling to keep most nodes active.

Table 8 makes the node-type composition evidence explicit and reveals a mixed rather than uniformly positive picture. On MUTAG class 1 (Non-Mutagen), the 1-2-GNN is the clearest success case: it generates C/O-dominated graphs with 80.0% C and 17.1% O, much closer to the real class distribution than the 1-GNN, which introduces atypical iodine (7.7%) and fluorine (2.0%). On MUTAG class 0 (Mutagen), however, the 1-2-GNN over-emphasises halogens (26.7% F, 11.0% Br, 3.6% I) and underproduces N, whereas the 1-GNN stays closer to the real C/N/O proportions. On PROTEINS class 0, the 1-2-GNN closely matches the real secondary-structure distribution (H: 50.9% vs. 50.4%, S: 47.1% vs. 46.5%, C: 2.0% vs. 3.1%). On PROTEINS class 1, by contrast, both models overproduce sheets and underproduce helix/coil, with the 1-2-GNN class 1 generator being the least faithful (88.9% S vs. 56.2% real).

Table 7: Structural comparison: 500 generated vs. real graphs per configuration. $D_{\mathrm{KS}}^{\mathrm{deg}}$ and $D_{\mathrm{KS}}^{\mathrm{size}}$ denote Kolmogorov–Smirnov statistics on degree and graph-size distributions, respectively. $\bar{d}$ = mean degree, $\bar{n}$ = mean graph size.

Configuration	$D_{\mathrm{KS}}^{\mathrm{deg}} \downarrow$	$D_{\mathrm{KS}}^{\mathrm{size}} \downarrow$	$\bar{d}_{\mathrm{real}}$ / $\bar{d}_{\mathrm{gen}}$	$\bar{n}_{\mathrm{real}}$ / $\bar{n}_{\mathrm{gen}}$
MUTAG				
1-GNN class 0	0.120	0.891	2.09 / 2.06	13.9 / 23.8
1-GNN class 1	0.094	0.893	2.24 / 2.23	19.8 / 27.3
1-2-GNN class 0	0.125	0.881	2.09 / 2.05	13.9 / 23.1
1-2-GNN class 1	0.155	0.594	2.24 / 2.23	19.8 / 24.3
PROTEINS				
1-GNN class 0	0.149	0.660	3.80 / 3.81	48.7 / 46.6
1-GNN class 1	0.213	0.537	3.64 / 3.73	23.7 / 24.0
1-2-GNN class 0	0.178	0.704	3.80 / 3.81	48.7 / 50.0
1-2-GNN class 1	0.165	0.738	3.64 / 3.64	23.7 / 29.5

A complementary graph-level view comes from normalized node-type entropy,

$$H_{\mathrm{norm}}(G) = \frac{-\sum_t p_t(G) \log p_t(G)}{\log K},$$

where $p_t(G)$ is the proportion of node type t in graph G and K is the number of node types in the dataset. Lower values mean that the graphs are more compositionally homogeneous, while higher values mean that the node types are more mixed. Figure 7 shows that these entropy shifts are again class-dependent. The clearest low-entropy case is PROTEINS Enzyme, where the 1-2-GNN generated graphs have much lower median entropy than both the real graphs (0.303 vs. 0.538) and the 1-GNN graphs (0.303 vs. 0.525), which

Table 8: Population-level node-type composition (%) for the 500-graph comparison. Real distributions are repeated within each class to make within-class comparisons direct.

Dataset / class	Model	Real	Generated
MUTAG c0	1-GNN	C 64.2, N 13.0, O 18.8, F 0.9, I 0.0, Cl 3.0, Br 0.2	C 57.2, N 10.9, O 17.0, F 6.2, I 0.0, Cl 8.6, Br 0.0
MUTAG c0	1-2-GNN	C 64.2, N 13.0, O 18.8, F 0.9, I 0.0, Cl 3.0, Br 0.2	C 45.8, N 0.1, O 12.8, F 26.7, I 3.6, Cl 0.0, Br 11.0
MUTAG c1	1-GNN	C 73.2, N 9.3, O 17.1, F 0.1, I 0.0, Cl 0.2, Br 0.0	C 63.4, N 7.6, O 19.3, F 2.0, I 7.7, Cl 0.0, Br 0.0
MUTAG c1	1-2-GNN	C 73.2, N 9.3, O 17.1, F 0.1, I 0.0, Cl 0.2, Br 0.0	C 80.0, N 2.9, O 17.1, F 0.0, I 0.0, Cl 0.0, Br 0.0
PROTEINS c0	1-GNN	H 50.4, S 46.5, C 3.1	H 54.3, S 45.7, C 0.0
PROTEINS c0	1-2-GNN	H 50.4, S 46.5, C 3.1	H 50.9, S 47.1, C 2.0
PROTEINS c1	1-GNN	H 40.7, S 56.2, C 3.1	H 24.5, S 75.5, C 0.0
PROTEINS c1	1-2-GNN	H 40.7, S 56.2, C 3.1	H 11.0, S 88.9, C 0.0

matches the strongly sheet-dominated explanation family. On MUTAG Non-Mutagen the same pattern appears more moderately (median 0.301 for 1-2-GNN vs. 0.545 for 1-GNN). On the other hand, the 1-2-GNN gives *higher* entropy than the 1-GNN on MUTAG Mutagen and PROTEINS Non-Enzyme. So the higher-order model does not simply make the generated graphs more homogeneous in every case. It pushes the generator toward different composition patterns depending on the class.

5.6.2 Cross-Model Classification

The most revealing analysis classifies each set of 500 generated graphs with *both* k-GNN classifiers, testing whether explanations produced by one model are recognisable by the other. Table 9 reports target class agreement rates, and Figure 8 visualises the full classification breakdown.

On MUTAG, three of four configurations show high cross-model agreement: 1-GNN class 0 at 90.4%, 1-GNN class 1 at 91.6%, and 1-2-GNN class 0 at 100%. The exception is striking. The 1-2-GNN's Non-Mutagen graphs (class 1) achieve 100% same-model agreement but only 12.4% cross-model agreement. The 1-GNN classifies 87.6% of them as *Mutagen*. A cautious interpretation is that the 1-2-GNN is using pairwise co-occurrence patterns within the carbon scaffold, or relative placement of small N/O motifs, rather than the node-level cues that dominate the 1-GNN prototypes. We cannot identify the exact pair features from these experiments alone, but the flip strongly suggests that the higher-order model's Non-Mutagen boundary depends on structural relationships the 1-GNN does not encode.

On PROTEINS, all four generators succeed at their target class (same-model agreement $\geq$ 98.4%), enabling a clean cross-model comparison. The most revealing asymmetry is on class 0 (Non-Enzyme). 1-GNN-generated Non-Enzyme graphs achieve 100% same-model agreement but only 9.4% cross-model agreement. The 1-2-GNN classifies 90.6% of them as Enzyme. In contrast, 1-2-GNN-generated Non-Enzyme graphs achieve 98.4% same-model agreement *and* 100% cross-model agreement. The 1-GNN also accepts them as Non-Enzyme. On class 1 (Enzyme), by contrast, both models largely agree (98–100% cross-model), suggesting that some class-defining protein patterns are visible to both architectures. Across the two datasets, the asymmetry is therefore class-dependent rather than uniform: transfer breaks down most strongly when one model has collapsed onto a

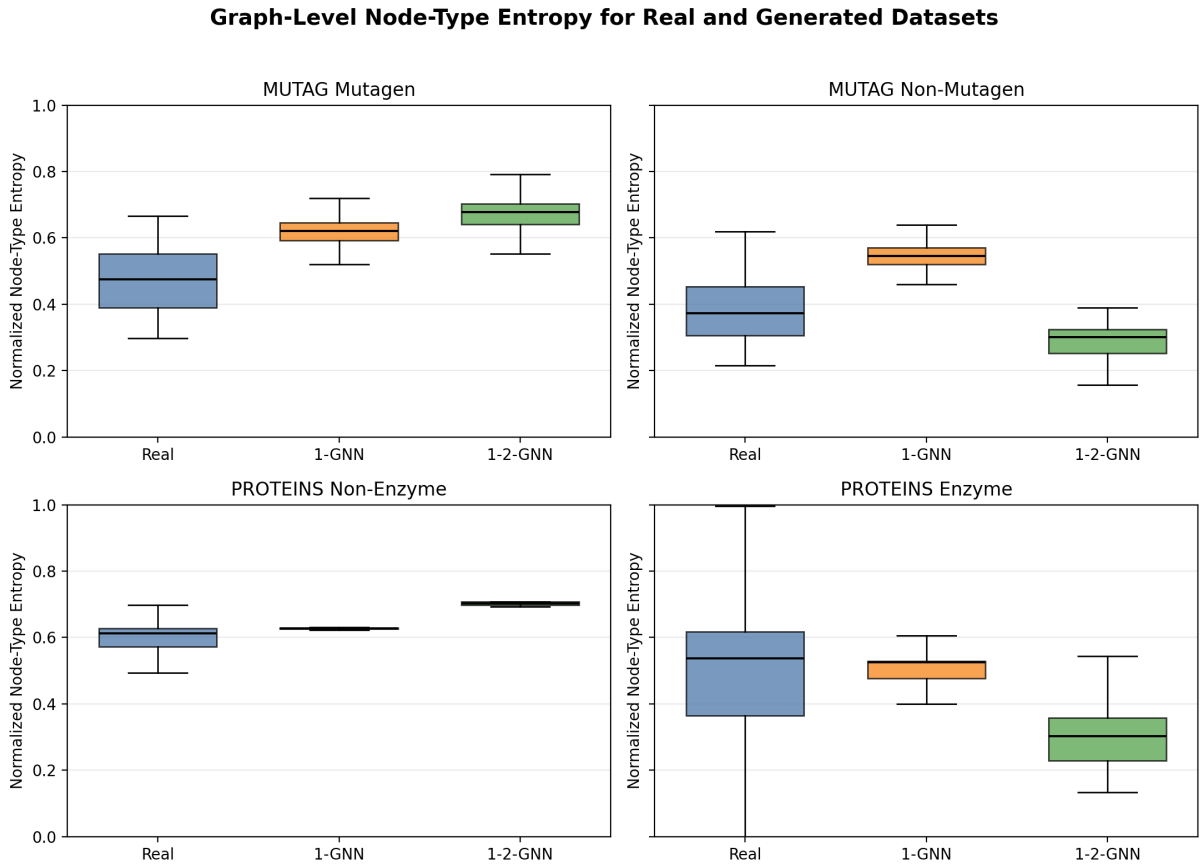
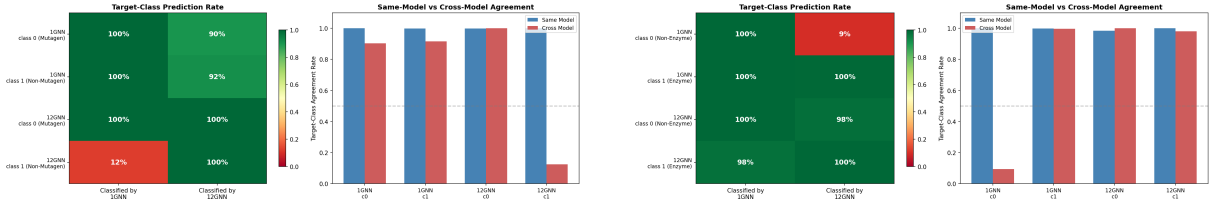


Figure 7: Graph-level normalized node-type entropy for the real training subsets and the generated datasets. Lower entropy indicates more homogeneous node-type composition within each graph. The strongest drop appears for PROTEINS Enzyme under the 1-2-GNN, while the other classes show different shifts rather than one uniform trend.

narrower, model-specific prototype family, and it remains high where the class structure seems easier or more shared across architectures.

Table 9: Cross-model classification of generated graphs (500 per class). Same = target class agreement by the generating model's classifier; Cross = agreement by the other model's classifier.

Generated by	Target class	Same	Cross
MUTAG			
1-GNN	Mutagen (c0)	100%	90.4%
1-GNN	Non-Mutagen (c1)	99.8%	91.6%
1-2-GNN	Mutagen (c0)	99.8%	100%
1-2-GNN	Non-Mutagen (c1)	100%	12.4%
PROTEINS			
1-GNN	Non-Enzyme (c0)	100%	9.4%
1-GNN	Enzyme (c1)	99.8%	99.6%
1-2-GNN	Non-Enzyme (c0)	98.4%	100%
1-2-GNN	Enzyme (c1)	100%	98.0%



(a) MUTAG (b) PROTEINS

Figure 8: Cross-model classification of generated graphs. Rows correspond to generator/class combinations and columns to the classifier used for evaluation. Large same-model bars together with sharply lower cross-model bars identify model-specific explanation families rather than universally shared prototypes.

5.6.3 Embedding Space Structure

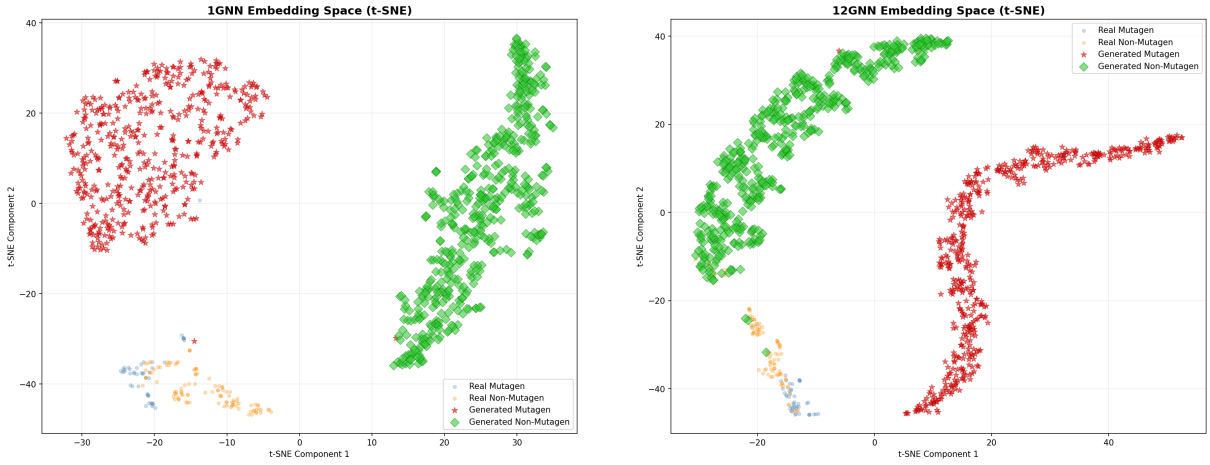
Figures 9 and 10 show t-SNE projections of real and generated graph embeddings in each model's learned representation space. To complement the visualisations, Table 10 reports the displacement of the generated class centroid from the real class centroid, normalised by the real between-class distance *within the same model's embedding space*.

On MUTAG (Figure 9), the generated graphs are cleanly separated by class, but they do not simply fill in the real-data clouds. Instead, each model produces two large generated manifolds that sit away from the much smaller real clusters. The same-model agreement is therefore not coming from faithful sampling of the empirical distribution, but from the generator finding class-consistent regions of embedding space. The normalized drift values support the class-specific picture: the 1-2-GNN drifts much less on Non-Mutagen than on Mutagen (3.93 vs. 17.02), matching its cleaner class-1 population statistics and its much less faithful halogen-heavy class-0 family.

The effect is even stronger on PROTEINS (Figure 10). In the 1-GNN space, the generated Non-Enzyme and Enzyme graphs occupy detached arcs and islands that only partially overlap the real distribution. In the 1-2-GNN space, the generated graphs collapse into several very tight disconnected prototype islands. The centroid-drift statistic should be read carefully here: it measures the location of the generated cloud mean, not its spread. This matters on PROTEINS 1-GNN class 1, where the drift is small (0.38) even though per-graph cosine similarity remains low, implying a dispersed cloud centered near the real class rather than faithful pointwise reproduction. Taken together, the t-SNE plots and drift values support a cautious interpretation of the population-level results: the generators preserve class-discriminative structure well enough to achieve high same-model agreement, but they do not reproduce the full geometry of the real dataset. The generated graphs are best interpreted as model-specific prototypes rather than faithful random samples.

Table 10: Normalized embedding centroid drift, defined within each model's embedding space as $\|\mu_{\text{real},c} - \mu_{\text{gen},c}\| / \|\mu_{\text{real},0} - \mu_{\text{real},1}\|$. Smaller values indicate that the generated class mean lies closer to the real class mean. These values are comparable within a model, but not across models.

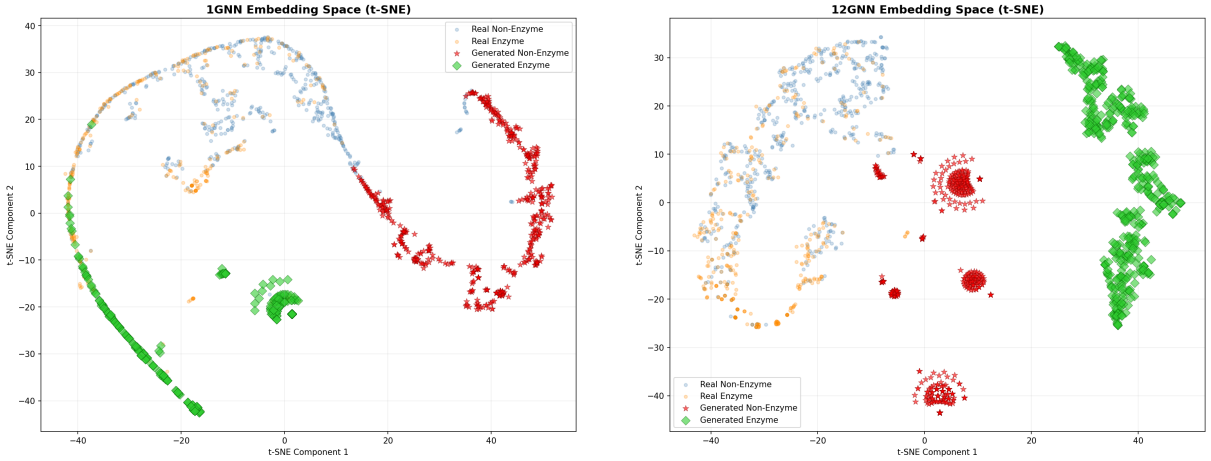
Configuration	Class 0 drift	Class 1 drift
MUTAG 1-GNN	5.99	10.19
MUTAG 1-2-GNN	17.02	3.93
PROTEINS 1-GNN	3.51	0.38
PROTEINS 1-2-GNN	1.18	7.02



(a) 1-GNN embedding space

(b) 1-2-GNN embedding space

Figure 9: t-SNE projections of MUTAG real (small dots) and generated (large markers) graph embeddings. The generated Mutagen and Non-Mutagen sets form large detached manifolds rather than filling the small real-data clusters, consistent with prototype generation rather than faithful sampling.



(a) 1-GNN embedding space

(b) 1-2-GNN embedding space

Figure 10: t-SNE projections of PROTEINS real (small dots) and generated (large markers) graph embeddings. The generated graphs form detached arcs and tight islands, especially in the 1-2-GNN space, showing that class-consistent generation does not imply full reproduction of the real embedding geometry.

6 Discussion

6.1 Claim 1: The Two Architectures Learn Different Prototype Families

The central claim supported by the experiments is not that the 1-2-GNN is better, but that it learns *different* class prototypes from the 1-GNN. The strongest evidence for this claim is the cross-model classification experiment in Section 5.6. If the two classifiers had learned essentially the same class representation, then graphs generated to satisfy one model should usually transfer to the other. That is not what we observe. On MUTAG, the 1-2-GNN's Non-Mutagen generator achieves 100% same-model agreement but only 12.4% cross-model agreement; the 1-GNN classifies 87.6% of those graphs as Mutagen. On PROTEINS, the asymmetry appears in the opposite direction on class 0: 1-GNN-generated Non-Enzyme graphs achieve 100% same-model agreement but only 9.4% cross-model agreement under the 1-2-GNN.

This inferential step needs some care. Low cross-model transfer on its own could reflect a weak or unstable generator. We read it instead as evidence of different learned prototypes because the low-transfer cases are not low-confidence failures under the generating model: they still have very high same-model agreement, clear within-model class separation in embedding space, and coherent population structure. In other words, the generator is not just producing random off-manifold graphs. It is producing graphs that satisfy one classifier's notion of the target class and that are often rejected by the other classifier. Under the present setup, that is the clearest sign that the two architectures rely on different class-defining regularities.

This reading is also consistent with the architecture. The hierarchical 1-2-GNN does not classify graphs from node marginals alone: its graph embedding concatenates the node-level and pair-level readouts (Equations 8 and 11), so pairwise co-occurrence patterns can affect the final decision boundary. This does not prove that the cross-model asymmetries

come from specific pair features. Direct attribution on the 2-GNN component would still be needed for that. But it does give a plausible reason why two models with similar test accuracy can still settle on different prototype families.

The qualitative examples support the same interpretation. On MUTAG class 0, the 1-GNN repeatedly produces C/N/O-rich mutagenic scaffolds, whereas the 1-2-GNN shifts toward halogen-heavy families with almost no nitrogen in the top-ranked set. On MUTAG class 1, the 1-GNN favours oxygen- and iodine-bearing branches while the 1-2-GNN concentrates on compact carbon-dominated structures with small N/O motifs. On PROTEINS, the 1-GNN and 1-2-GNN both separate Enzyme from Non-Enzyme, but the best explanations emphasise different secondary-structure compositions and different levels of regularity. Taken together, the figures and cross-model asymmetries support a common conclusion: higher-order message passing changes *what* the model treats as a representative class member.

6.2 Claim 2: Higher-Order Message Passing Is Not Uniformly Better

The second claim is more limited. The results do not support a blanket superiority statement for the 1-2-GNN, either in accuracy or in explanation quality. Instead, they show selective gains and selective failures. On MUTAG, the 1-2-GNN gives the stronger result on Non-Mutagen: its generated node-type distribution is much closer to the real class than the 1-GNN's, and its class-1 centroid drift is also smaller. On the same dataset's Mutagen class, however, the pattern reverses: the 1-2-GNN overproduces fluorine, bromine, and iodine and diverges more strongly from the real C/N/O composition than the 1-GNN.

The same mixed picture appears on PROTEINS. On Non-Enzyme, the 1-2-GNN gives the closest population-level node-type match to the real data, while the 1-GNN achieves the highest original validation score. On Enzyme, the 1-2-GNN attains much higher validity (94% vs. 37%), but its embedding similarity is even lower than the 1-GNN's ($s = 0.151$ vs. 0.356), its generated graphs become strongly sheet-heavy relative to the real class, and its graph-level entropy collapses markedly (Figure 7). The most defensible interpretation is therefore case-specific rather than global: higher-order message passing can yield clearer fidelity gains, but only for some classes and by some metrics.

This is why the cross-model asymmetries should be read as evidence of *difference* more clearly than evidence of *dominance*. The 1-2-GNN does not consistently outperform the 1-GNN across MUTAG and PROTEINS, and the 1-GNN is not simply a weaker version of the same representation. Each architecture appears to emphasise different structural cues, and those cues are more or less helpful depending on the class being modelled.

6.3 Claim 3: The Generated Graphs Are Prototypes Rather Than Faithful Samples

The third claim concerns what kind of explanation GIN-Graph is actually producing in this setting. The generated graphs preserve class-discriminative structure, but they do not reproduce the empirical data distribution closely enough to be interpreted as faithful random samples. The population-level comparison in Section 5.6 makes this visible. Across both datasets, the generators match average degree far more reliably than

graph size. On MUTAG, all four configurations remain substantially oversized relative to the real graphs. On PROTEINS, size regularisation helps, but large mismatches remain, especially for the 1-2-GNN Enzyme generator.

The embedding-space analysis points in the same direction. In both datasets, the generated graphs form detached manifolds or tight prototype islands rather than filling the real-data clouds. This means that high same-model agreement is not coming from faithful reproduction of the observed population. It is coming from the generator finding regions of each classifier's embedding space that are decisively class-consistent. The generated graphs are therefore best interpreted as *architecture-specific prototypes*: they expose what each classifier treats as a convincing class exemplar, even when that exemplar lies away from the center of the empirical data cloud.

This reading also clarifies the granularity issue. Most explanations have $\kappa \approx 0$ and remain close to full-graph templates rather than sparse substructures. That is a structural property of the method. GIN-Graph is answering "what does a typical class member look like according to this model?" rather than "which local subgraph caused this prediction?" In that sense, the coarse granularity is not a defect in the conclusion but part of what the method is designed to reveal.

6.4 What Would Change This Interpretation?

The three claims above are supported by the present evidence, but they are not beyond revision. Several future results would weaken or overturn them. First, if repeated runs across multiple seeds and data splits showed that the large cross-model asymmetries disappear or vary erratically, then the claim of architecture-specific prototype families would be much less secure. Second, if a higher-order–aware generator removed the current generator–classifier mismatch and the two models then produced strongly convergent explanation populations, the present differences could reasonably be reinterpreted as artefacts of optimisation rather than differences in learned representation. Third, if richer structural metrics such as motif counts or subgraph frequencies showed that the generated populations track the real datasets much more closely than the current degree/size/embedding analyses suggest, then the "prototype rather than sample" reading would need to be softened. The current conclusion is therefore best understood as a constrained one: under the present generator, datasets, and single-seed evaluation, higher-order message passing changes the learned prototypes, but does not establish universal superiority.

7 Limitations

Several limitations qualify our findings:

Single seed, no error bars. All experiments use a single data split (seed 42) without cross-validation. With only 38 test graphs on MUTAG, the identical 89.5% best accuracy for both models could be coincidence. A difference of one misclassified graph changes accuracy by 2.6 percentage points. The generated-sample counts are also modest: for example, MUTAG 1-GNN class 0 has 36% validity from $n = 50$, which corresponds to an approximate Wilson 95% interval of 24–50%, while MUTAG 1-2-GNN class 1 has

82% validity from $n = 100$, corresponding to roughly 73–88%. These intervals capture only sampling uncertainty within a run; seed-to-seed and split-to-split variation remain unmeasured. This is why we treat the qualitative divergence in generated explanations as stronger evidence than small within-table percentage gaps.

No cross-validation. Best test accuracy is reported as the maximum over epochs (early stopping by oracle), which overestimates generalisation performance. A more rigorous evaluation would use a held-out validation set for model selection.

Classifier–generator coupling. GIN-Graph explanations are shaped by how the classifier's internal representations interact with the generator's optimisation landscape. The 1-2-GNN class 0 (Non-Enzyme) generator on PROTEINS achieves moderate prediction probability ($p = 0.564$), lower than the 1-GNN's near-perfect $p = 0.986$ on the same class. We attribute this gap to an *embedding dimensionality mismatch*: the 1-2-GNN classifier operates on a concatenated embedding $\mathbf{h}_G = [\mathbf{h}_G^{(1)} \| \mathbf{h}_G^{(2)}] \in \mathbb{R}^{128}$, double the 1-GNN's $\mathbb{R}^{64}$, while the generator uses the same architecture in both cases. The 2-GNN component's response to adjacency changes is mediated by pair-level aggregation (Equations 27–28), creating a more complex gradient landscape. However, with graph size and node type regularisation, the generator achieves sufficient prediction probability for meaningful cross-model analysis (98.4% same-model agreement).

The 1-GNN class 1 (Enzyme) generator (37% validity, $s = 0.356$) exhibits a different pattern: $p = 0.919$ indicates the generator can find the target class region, but the generated graphs diverge from the real Enzyme embedding centroid. This is a fidelity problem (the generated graphs are atypical Enzyme structures) rather than a guidance problem (the classifier does recognise them as Enzyme).

Generator–classifier output space mismatch. A fundamental consideration is that GIN-Graph was designed for standard (1-WL) GNNs, and its generator architecture reflects this. The generator outputs an adjacency matrix $\tilde{A}$ and node feature matrix $\tilde{X}$, quantities that a 1-GNN directly consumes. A 1-GNN's decisions are based on node features and local neighbourhoods, which the generator can target directly: "place nitrogen here, connect it to these carbons." The 1-2-GNN, however, makes decisions in a pairwise feature space derived from all $\binom{n}{2}$ node pairs. The generator has no mechanism to directly control pair-level representations. It can only influence them indirectly through edge and node choices. Despite this, the generator achieves high same-model agreement for all configurations ($\geq 94\%$) after structural regularisation, demonstrating that the indirect optimisation path is viable. However, the consistently lower prediction probability for the 1-2-GNN ($p = 0.564$ vs. 0.986 on PROTEINS class 0) suggests that a higher-order-aware generator, one that directly produces pair-level features, could improve results further.

Evaluation metrics. The validation score $v = (s \cdot p \cdot d)^{1/3}$ can be dominated by a single low component. The degree score, which measures structural realism via average degree alone, is a coarse proxy that does not capture finer structural properties like motif frequencies or degree distributions. The embedding similarity term s is also computed in each model's *own* embedding space, so raw 1-GNN and 1-2-GNN values are informative within model but not strictly apples-to-apples across models. The normalized centroid-drift statistic in Table 10 has the same limitation and, additionally, captures the generated cloud mean rather than within-cloud spread.

No baselines. We do not compare with model-level XGNN [Yuan et al.(2020)]. We also do not compare with instance-level methods such as GNNExplainer [Ying et al.(2019)]

and PGExplainer [Luo et al.(2020)]. These methods target different explanatory granularities and are therefore not direct plug-in substitutes for GIN-Graph, but a side-by-side comparison would still contextualise what is gained and lost by model-level prototype generation.

8 Future Work

Several directions could strengthen this investigation:

- **Higher-order–aware generator.** The most impactful extension would be a generator architecture designed for k-GNN classifiers. Rather than outputting only adjacency and node features, such a generator could directly produce pair-level (or k-set–level) features, or use higher-order message passing internally so that it "thinks" in the same representation space as the classifier. This would disentangle the question of whether the 1-2-GNN learns richer structural patterns from the confound of generator–classifier mismatch identified in our limitations.
- Use cross-validation and multiple random seeds for robust accuracy and explanation quality estimates. The current single-seed results, while consistent across configurations, may not generalise.
- Investigate *why* the 1-GNN and 1-2-GNN learn different class representations—e.g., which specific pairwise patterns drive the 1-2-GNN's more selective Non-Enzyme boundary—using attention or gradient-based attribution on the 2-GNN component.
- Extend to the 1-2-3-GNN architecture, which adds 3-set (triplet) message passing for even higher expressiveness, to test whether the interpretability benefit continues up the k-WL hierarchy.
- Incorporate richer structural evaluation metrics beyond average degree, such as clustering coefficients, motif frequencies, or subgraph counts, to better capture the qualitative differences between generated explanations.
- Apply the framework to larger and more diverse datasets where the expressiveness gap between 1-WL and higher-order tests is more pronounced.

9 Conclusion

We introduced two technical pieces needed to study higher-order interpretability with GIN-Graph: a fully differentiable dense wrapper for hierarchical k-GNNs, and a corrected validation metric that measures actual embedding similarity rather than reusing prediction probability as a proxy. These are enabling contributions: they make a direct comparison between 1-GNN and 1-2-GNN explanations possible and make the resulting quantitative evaluation more trustworthy.

With those pieces in place, the main empirical conclusion is that the two architectures learn different class prototype families rather than a single shared representation. The strongest evidence comes from cross-model classification. Graphs generated to satisfy one model are sometimes accepted almost perfectly by that model but rejected by the other, showing that similar test accuracy does not imply similar internal class representations. The qualitative examples and population-level comparisons support the same reading on both MUTAG and PROTEINS.

The higher-order model is not uniformly better. Its gains are selective: it gives the clearest fidelity improvements on some classes, but it is worse on others, and the mixed metric profiles rule out a blanket superiority claim. The generated graphs should therefore be interpreted as architecture-specific prototypes rather than faithful samples of the real data distribution. After adapting GIN-Graph to the hierarchical 1-2-GNN and correcting the validation metric, we find that 1-GNN and 1-2-GNN learn different class prototype families. Higher-order message passing changes *what* is represented, not uniformly *how well*.

References

[Borgwardt et al.(2005)] K. M. Borgwardt, C. S. Ong, S. Schönauer, S. V. N. Vishwanathan, A. J. Smola, and H.-P. Kriegel. Protein function prediction via graph kernels. *Bioinformatics*, 21(suppl_1):i47–i56, 2005.

[Cai et al.(1992)] J.-Y. Cai, M. Fürer, and N. Immerman. An optimal lower bound on the number of variables for graph identification. *Combinatorica*, 12(4):389–410, 1992.

[Debnath et al.(1991)] A. K. Debnath, R. L. Lopez de Compadre, G. Debnath, A. J. Shusterman, and C. Hansch. Structure-activity relationship of mutagenic aromatic and heteroaromatic nitro compounds. *J. Med. Chem.*, 34(2):786–797, 1991.

[Gulrajani et al.(2017)] I. Gulrajani, F. Ahmed, M. Arjovsky, V. Dumoulin, and A. Courville. Improved training of Wasserstein GANs. In *NeurIPS*, 2017.

[Jang et al.(2017)] E. Jang, S. Gu, and B. Poole. Categorical reparameterization with Gumbel-softmax. In *ICLR*, 2017.

[Luo et al.(2020)] D. Luo, W. Cheng, D. Xu, W. Yu, B. Zong, H. Chen, and X. Zhang. Parameterized explainer for graph neural network. In *NeurIPS*, 2020.

[Morris et al.(2019)] C. Morris, M. Ritzert, M. Fey, W. L. Hamilton, J. E. Lenssen, G. Rattan, and M. Grohe. Weisfeiler and Leman go neural: Higher-order graph neural networks. In *AAAI*, 2019.

[Sun et al.(2025)] J. Sun, S. Pei, F. Zhang, and N. V. Chawla. GIN-Graph: A generative interpretation network for model-level explanation of graph neural networks. *Neurocomputing*, 2025.

[Ying et al.(2019)] R. Ying, D. Bourgeois, J. You, M. Zitnik, and J. Leskovec. GNNExplainer: Generating explanations for graph neural networks. In *NeurIPS*, 2019.

[Yuan et al.(2020)] H. Yuan, J. Tang, X. Hu, and S. Ji. XGNN: Towards model-level explanations of graph neural networks. In *KDD*, 2020.